Journal of King Saud University Computer and Information Sciences (2026) 38:738
https://doi.org/10.1007/s44443-026-01127-1

RESEARCH

DMAE: dual-memory active ensemble learning for multi-class imbalanced concept-drifting data streams

Meng Han[1] · Yajie Xue[1] · Yikai Li[1] · Cuicui Ma[1] · Jian Ding[1] · Juan Li[1]

Received: 30 April 2026 / Accepted: 21 July 2026
© The Author(s) 2026

Abstract

Multiclass imbalance and concept drift often coexist in real-world data streams, and the challenge becomes more severe under limited labeling budgets, where existing online ensemble and active learning methods still struggle to preserve minority-class recognition while adapting to evolving concepts. To address this issue, we propose DMAE, a dual-memory active ensemble learning method for multiclass imbalanced concept-drifting data streams. DMAE integrates an ensemble classifier, a drift detector, an instance sliding window, a label sliding window, a long-term prototype memory, and an initialization training sequence into a unified online framework for prediction, querying, memory maintenance, and model updating. To improve label efficiency, we develop a variable-threshold uncertainty strategy based on a decomposable asymmetric margin-threshold matrix, which combines a global threshold with class-pair-level corrections to focus queries on genuinely ambiguous, minority-relevant regions while controlling annotation cost. We further design drift-strength-aware dual-memory initialization (SDMI) and a probationary soft-replacement strategy (PDSR), which adaptively balance recent information and long-term prototypes according to online-estimated drift strength, supporting both historical-pattern retention and rapid adaptation under abrupt, gradual, and mixed drifts. We also propose a composite sample-weighting formulation, PCN-Weight, which jointly models boundary difficulty, class-imbalance status, sample–prototype relations, and temporal decay to guide incremental training and strengthen minority-class and hard-region representations. Experiments on 15 synthetic data streams and 5 real-world imbalanced data streams show that DMAE achieves more stable overall performance than ten state-of-the-art ensemble baselines in Accuracy, Kappa, G-Mean, and Recall, while remaining robust under different labeling rates and key hyperparameter settings.

Keywords Online active learning · Multiclass imbalanced data streams · Concept drift adaptation · Active ensemble learning · Asymmetric uncertainty · Dual-memory initialization

1 Introduction

Multiclass imbalanced data streams often evolve over time, so the learner must cope with concept drift while maintaining reliable recognition of minority classes. In such non-stationary environments, drift may progressively invalidate previously learned decision boundaries and also reduce the effectiveness of active querying, because the most informative samples, class proportions, and confusion patterns all change over time. The problem becomes even more challenging under limited labeling budgets, where only a small fraction of arriving instances can be annotated and the resulting supervision is easily biased toward majority classes or currently ambiguous regions. Therefore, an effective online learner should jointly address drift adaptation,

✉ Yajie Xue
1499809229@qq.com

Meng Han
17747843744@163.com

Yikai Li
997786907@qq.com

Cuicui Ma
ma07972025@163.com

Jian Ding
49807889@qq.com

Juan Li
compute2006_2@126.com

[1] North Minzu University, Yinchuan, China

Published online: 21 August 2026

multiclass imbalance, and budget-aware label acquisition within a unified framework.

The joint presence of concept drift and multiclass imbalance can substantially undermine the generalization ability of stream classifiers, and their interaction often leads to more severe and persistent performance degradation than either factor alone. For example, in imbalanced data-stream scenarios, many concept drift detectors tend to over-rely on the overall error rate, being more sensitive to changes in the majority class while responding sluggishly to abrupt distributional shifts within minority classes, thereby yielding drift-detection results biased toward the majority class (Korycki and Krawczyk 2021). Similarly, traditional imbalance-handling strategies based on oversampling or undersampling often fail in drift scenarios with variable imbalance ratios, because such methods typically rely on static training assumptions and struggle to track in a timely manner the coevolution of class priors and class-conditional distributions. Although prior studies have explored imbalance mitigation, drift adaptation, online ensemble learning, and active querying from different perspectives, most of them address only part of the target problem and still lack a unified treatment of multiclass imbalance, mixed concept drift, constrained labeling budgets, and stable online ensemble evolution. In particular, the coupling among query bias, imbalance estimation, drift response, and minority-class preservation remains insufficiently addressed in current approaches. Moreover, most existing studies implicitly assume that true labels can be continuously and sufficiently obtained during training, an assumption that often does not hold in real-world scenarios with high labeling costs and limited resources. When the labeling budget is constrained, the superposition of multiclass imbalance and concept drift makes online active learning even more challenging, because the model can obtain supervision only from a small number of selected queried samples, while a large proportion of arriving instances remain unlabeled, thereby further exacerbating the difficulty of imbalance estimation and drift adaptation. To the best of our knowledge, an efficient unified method that simultaneously addresses these challenges under limited labeling conditions is still lacking.

To address the above challenges, this paper proposes a dual-memory active ensemble learning method, DMAE, for multiclass imbalanced concept-drifting data streams, aiming to simultaneously enhance minority-class recognition capability, drift adaptability, and overall stability under limited labeling budgets. The overall framework consists of an online ensemble classifier composed of multiple base classifiers, an ADWIN-based drift detection module, a label sliding window for approximately unbiased estimation of the current class distribution, class-wise short-term sample windows and long-term prototype memory, and an initialization training sample sequence used to reconstruct new base classifiers when drift occurs. On this basis, we propose an uncertainty modeling and hybrid labeling mechanism based on decomposable asymmetric margins, through the collaboration of random labeling and BAM^2-Lite-driven active labeling: on the one hand, the random subprocess provides approximately unbiased multiclass labeled samples for the label sliding window; on the other hand, the active subprocess adaptively adjusts the margin threshold at the class-pair scale, concentrating the limited labeling budget on genuinely ambiguous boundary regions relevant to minority classes. Subsequently, we design a drift-strength-aware dual-memory initialization and soft replacement mechanism, SDMI-PDSR, which simultaneously maintains a short-term sample window and a long-term prototype memory for each class, and uses the drift-strength index provided by ADWIN to adaptively control the mixing ratio of the two in the initialization training sequence. It then determines whether to retain or discard candidate base classifiers through performance-and-diversity utility evaluation within a limited trial period, thereby balancing historical pattern inheritance and rapid adaptation to new concepts under abrupt, gradual, and hybrid drift. Finally, we propose a prototype-contrast and class-quota-normalized comprehensive sample-weighting formulation, PCN-Weight, which unifies decision-boundary difficulty, the dynamically imbalanced state estimated by the label sliding window, the geometric relationship between samples and intra-class prototypes, and time decay into a single training weight. In this way, the allocation of training intensity during online bagging updates is regulated more finely along both the temporal axis and the class axis, thereby improving the utilization efficiency of limited labeled samples and strengthening the representation quality of minority classes and locally difficult regions.

The main contributions of this work are as follows:

(1) A dual-memory active ensemble framework termed DMAE is proposed for multiclass imbalanced concept-drifting data streams. It systematically integrates an online ensemble classifier, a drift-detection module, a label sliding window for approximately unbiased estimation of the current class distribution, and class-wise short-term sample windows and long-term prototype memory into a unified online architecture. The resulting framework enables collaborative modeling of prediction, label querying, memory maintenance, and model-structure updating under the joint presence of fixed and variable multiclass imbalance ratios, virtual and real drift together with their hybrid forms, limited labeling budgets, and adaptive ensemble size.
(2) A decomposable asymmetric margin-threshold matrix and a corridor-gated threshold field (BAM^2-Lite) are developed to model class-pair asymmetry at a fine granularity. They jointly capture class potential, histori-

cal confusion patterns, and evolving imbalance structure to derive direction-aware threshold biases between class pairs. The adjustment is activated only within a predefined uncertainty corridor, while a conservative threshold cap prevents the effective query threshold from exceeding the global baseline. In this way, limited labeling budgets are concentrated on genuinely ambiguous and minority-relevant boundary instances without sacrificing the original safety margin.

(3) A drift-strength-aware dual-memory initialization scheme (SDMI) and a performance-and-diversity-driven soft replacement strategy (PDSR) are proposed to support robust ensemble evolution under concept drift. For each class, DMAE maintains a short-term sample window and a long-term prototype memory, and uses an online drift-strength estimate to adaptively set the mixing ratio when constructing the initialization sample sequence. A newly created base classifier is trained from this sequence and admitted as a trial member; it is retained only when its trial-period utility indicates improvement over at least one existing member. This mechanism mitigates cold-start effects, reduces excessive forgetting, and avoids abrupt ensemble resets across abrupt, gradual, and hybrid drift regimes.

(4) A prototype-contrast and class-quota-normalized comprehensive sample-weighting formulation (PCN-Weight) is constructed. It aggregates margin-based prediction difficulty, the imbalance degree estimated from the approximately unbiased label sliding window, the relative proximity between a sample and its intra-class prototypes, and a time-decay factor into a single scalar weight. The resulting weight emphasizes hard and minority-relevant samples locally, while imposing a soft quota constraint on the cumulative effective contribution of majority classes globally, thereby regulating incremental training intensity along both temporal and class dimensions.

The remainder of this paper is organized as follows. Section 2 reviews related work on active learning for data streams, imbalanced data-stream classification, concept drift adaptation and ensemble learning, and integrated methods under multiple challenges. Section 3 details the proposed DMAE framework and its key components, including the hybrid uncertainty-based querying strategy, the drift-strength-aware dual-memory initialization with soft replacement, and the PCN-Weight reweighting mechanism. Section 4 presents experimental settings, baseline methods, and evaluation protocols, and systematically verifies the effectiveness and robustness of DMAE on diverse synthetic and real-world data streams. Section 5 concludes the paper and outlines directions for future work.

2 Related work

2.1 Active learning for data streams

In label-scarce streaming scenarios, active learning improves model performance by selectively requesting true labels for only a small number of instances, and is therefore a key avenue for controlling annotation costs. Common strategies can be broadly grouped into three categories: uncertainty-based strategies (Nguyen et al. 2022), which select instances according to predictive uncertainty or distance to the decision boundary; random-sampling strategies (Zhang et al. 2023), which draw instances more uniformly from the stream to preserve distributional representativeness; and hybrid strategies (Liu et al. 2021), which combine uncertainty and representativeness to better maintain accuracy and robustness under limited labeling budgets. Compared with traditional offline supervised learning, however, active learning in data streams must additionally account for continuous distribution evolution and delayed or partial supervision, which makes strategy design substantially more challenging.

One representative line of work embeds label querying directly into existing supervised or ensemble models. Salama et al. proposed an ensemble active learning model for heterogeneous data analytics (Salama et al. 2022), which actively selects a small number of high-value samples to reduce manual annotation cost and leverages ensemble learning to improve classification stability; however, it mainly targets static heterogeneous data and does not explicitly address continuous distribution evolution in streaming environments. Niu et al. proposed ADESSA (Niu et al. 2023), which combines margin sampling with democratic co-learning and balanced sampling to construct a limited-label training set from both manually labeled and automatically labeled high-confidence samples, but its effectiveness depends strongly on the reliability of pseudo-labels. Polyzos et al. proposed a weighted Gaussian-process ensemble framework (Polyzos et al. 2024), which improves sample-selection efficiency by adaptively weighting ensemble members and sampling criteria, yet it is still formulated for more general active learning settings rather than multiclass imbalanced drifting streams. Wang et al. proposed MOOB and MUOB (Wang et al. 2016), which improve performance on multiclass imbalanced data streams through resampling and online ensembles; nevertheless, if uncertainty-based querying is directly overlaid on top of such methods, the queried set may become systematically concentrated on hard-to-classify regions, thereby distorting the labeled subset used for distribution estimation. Park et al. proposed an active learning strategy for concept-drifting data streams (Park and Kang 2016),

which incorporates drift-severity assessment into incremental classification and dynamically adjusts querying probabilities; despite its low computational overhead, its effectiveness depends on the quality of drift estimation and may degrade under noisy or complex drift patterns.

Another line of work jointly designs the querying mechanism and the streaming learning framework at the system level. Guo et al. proposed SSALE-EIS (Guo et al. 2024), which integrates semi-supervised exploitation of unlabeled data and active querying of key samples within an online ensemble framework, but still needs to balance pseudo-label stability and querying efficiency during rapid distribution evolution. Halder et al. proposed AACE-DI (Halder et al. 2023), which combines clustering-based ensemble learning with automatic uncertainty adjustment to reduce expert labeling effort in drifting imbalanced streams, but its performance depends on the stability of clustering structure under drift. Zhang et al. proposed ROALE-DI (Zhang et al. 2022), which combines a long-term stable classifier with dynamic classifiers and updates member weights through reinforcement signals to improve minority-class sensitivity under drift and imbalance; however, the method remains sensitive to reward design and the stability of online reinforcement updates. Li et al. proposed AdaAL-MID (Han et al. 2024), which co-designs ensemble classification, drift detection, and hybrid querying for multiclass imbalanced drifting streams, but under strict labeling budgets it still faces nontrivial coordination among query frequency, class coverage, and model-update pace. Krawczyk et al. proposed EAL-MAB (Krawczyk and Cano 2019), which formulates querying decisions and strategy switching as a multi-armed bandit problem, yet its gains depend heavily on the diversity and complementarity of the ensemble members.

Overall, active learning for data streams has made substantial progress under labeling-budget constraints, including uncertainty-based querying, hybrid querying, and integration with online ensembles. However, when multiclass imbalance and hybrid concept drift coexist over long horizons, three structural limitations remain: querying decisions are still often driven by a single uncertainty signal, the bias introduced by active querying is frequently entangled with downstream statistical estimation, and existing mechanisms often struggle to simultaneously maintain minority-class protection, drift-response speed, and label-utilization efficiency.

2.2 Imbalanced data stream classification

Research on imbalanced data-stream classification initially focused mainly on binary-class settings (Salehi and Khedmati 2025; Zhai et al. 2022; Wang and wang 2023; Lu et al. 2017; Zhao et al. 2023), where the antagonistic relation between a single majority class and a single minority class is relatively easy to characterize. In multiclass settings, by contrast, whether a class behaves as a majority or a minority depends on its relative relation to other classes, and decomposing the task into multiple binary subtasks may lose important information about inter-class relativistic structure and transfer patterns (García et al. 2018). As a result, multiclass imbalance is substantially more difficult than binary imbalance, especially when minority-to-minority confusion and class-role switching must be considered simultaneously.

To improve minority-class learning in streaming environments, a large body of work has explored cost-sensitive learning, online resampling, and dynamically weighted ensembles. Wang et al. proposed OLI^2DS (You et al. 2023), which incrementally updates the classifier while integrating active querying and reweighting strategies to improve minority recognition; however, its performance can degrade when drift occurs frequently and the updating pace cannot keep up with rapid distributional changes. Du et al. proposed DyAbst (Krawczyk and Cano 2018), an online ensemble method that dynamically estimates misclassification costs, performs balanced sampling over incoming instances, and updates base-classifier weights, but its robustness depends strongly on the stability of cost estimation under rapid drift. Gao et al. proposed a dynamic selection ensemble framework (Jiao et al. 2024), which combines drift detection, subset selection, and resampling to alleviate imbalance; yet the subset-evaluation process may incur substantial overhead in multiclass settings. Lu et al. proposed DWMIL (Lu et al. 2017), which adjusts voting weights according to classifier performance and age and later introduced an adaptive-window extension, but its weight adaptation may still lag when class roles reverse. Lin et al. proposed ESOS-ELM (Mirza et al. 2015), which exploits historical information through resampling and a storage module, but requires prior knowledge of the imbalance ratio and is therefore less suitable for streams with rapidly changing class proportions. Krawczyk et al. proposed EOBag (Du et al. 2021), in which low-confidence learners abstain from voting and a dynamic threshold determines the participating member set; however, its effectiveness is sensitive to threshold adaptation and fluctuations in the effective voting pool. Jian et al. proposed OEC (Jian et al. 2024), which uses a normalized exponentiated-gradient scheme to combine multiple online classifiers and improve robustness under noisy labels, but its gains depend on the complementarity of the base learners and the alignment between weight adaptation and drift phases.

Overall, existing methods for imbalanced data-stream classification have improved minority-class learnability through reweighting, resampling, cost-sensitive learning, and online ensembles. Nevertheless, many of them are still designed under binary or fully supervised assumptions, and they do not explicitly provide a unified treatment of

multiclass imbalance relations, evolving class-pair confusion structure, and limited-label online updating.

2.3 Concept drift adaptation and ensemble learning

In data streams where concept drift and imbalance coexist, concept drift may appear together with changing class priors and evolving class-conditional distributions. Existing solutions can be broadly categorized into block-based schemes and instance-wise online schemes. Although both categories contribute to drift adaptation, they exhibit different limitations when multiclass imbalance, hybrid drift, and limited labeling budgets are present simultaneously.

Block-based approaches usually partition the stream into consecutive chunks, perform rebalancing or cost adjustment within each block, and combine block-level drift detection with ensemble updating. The Recursive Ensemble Approach (REA) (Chen and He 2011) introduces a distance metric to select minority-class samples from historical blocks and supplements them into the current block, while dynamically fusing historical hypotheses for prediction. ACDWM (Lu et al. 2020) dynamically adjusts block size and ensemble weights to balance stability and adaptivity under drift. HBBE (Mahdi et al. 2021) combines multiclass drift detection with a block-weighting mechanism to handle abrupt and gradual drift. WECOI (Czarnowski 2022) embeds imbalance handling into block construction and uses one-class decomposition together with oversampling and weighted ensemble replacement. WENNML (Wu et al. 2023), although designed for multi-label streams, also follows a sequential block-training strategy with weighted voting and interpretable updating. These methods provide important support for learning from streams where drift and imbalance coexist, but they remain largely driven by block-level statistics and block-level triggers. As a result, they are often insensitive to rapid within-block changes, may lag in updating under abrupt drift, and still rely heavily on sufficiently available labeled data within each block.

In contrast, instance-wise online methods adapt in real time through continual updating, allowing classifiers and ensemble weights to evolve together with the stream. This paradigm is better aligned with rapidly changing environments, but it also increases design complexity because drift adaptation, imbalance mitigation, and budgeted label acquisition may all need to be coordinated within the same online loop. ROALE-DI (Zhang et al. 2022), for example, integrates reinforcement-guided label selection and classifier updating to cope with drifting imbalanced streams under limited budgets, but is sensitive to reward design and policy stability. More broadly, many instance-wise approaches improve adaptability through continuous updating and dynamic selection, yet they still face challenges in computational efficiency, robust memory coordination, and stable adaptation under multiclass proportion shifts and accumulated noise.

Overall, block-based methods are effective when drift is relatively smooth and labels are sufficiently available, whereas instance-wise methods are generally more responsive to evolving distributions. However, under rapid within-block drift, multiclass imbalance, variable imbalance ratios, and strict labeling constraints, existing drift-adaptation and ensemble-learning mechanisms still struggle to jointly maintain minority-class protection, timely drift response, and stable model evolution.

2.4 Integrated methods under multiple challenges

In recent years, integrated studies on multiclass imbalanced data streams and concept drift have gradually increased, yet most existing methods still combine only a subset of the challenges involved. For instance, MOS-ELM (Mirza and Lin 2016) was among the earlier attempts to handle hybrid drift together with variable multiclass imbalance ratios, but its block-based processing paradigm cannot respond promptly to drift changes occurring within a single block. PEP (Hidalgo et al. 2021) adjusts ensemble diversity strength online through parameter estimation, aiming to maintain more stable component differences under drift; however, its focus remains largely on diversity regulation and does not explicitly characterize evolving multiclass imbalance ratios, minority-to-minority confusion patterns, or the risks associated with asymmetric costs. ROSE (Cano and Krawczyk 2022) uses online feature subsets, background ensembles, class-wise sliding windows, and self-adjusting bagging to improve robustness on multiclass imbalanced drifting streams, but it still mainly relies on window memory and resampling-based structural robustness, and lacks dedicated closed-loop mechanisms for query bias under limited labeling budgets. Incremental decision-tree drift-mitigation methods (Tarazodar et al. 2024) improve drift discovery and structural adaptation through subtree monitoring and incremental updates, but they are closer to a single-model continual-update paradigm and do not inherently provide ensemble-level diversity maintenance or stable fusion under multiclass imbalance. EDAC (Gong and Dong 2024) uses entropy differences as cues to balance incoming blocks, combines density sampling to distinguish minority-sample quality, and adopts self-feedback strategies for initializing ensemble weights, yet it still operates at the block level and thus remains vulnerable to delayed response under rapid within-block drift and class-role switching.

Taken together, prior studies have made meaningful progress in active querying, imbalance handling, drift adaptation, and online ensemble updating, but they still fall short of jointly addressing four requirements of the target scenario

considered in this work: direction-sensitive query control under multiclass imbalance, approximately unbiased estimation of evolving class distributions under active labeling, drift-aware memory reconstruction that balances recent and long-term information, and stable candidate-classifier admission and replacement within an online ensemble. These limitations motivate the DMAE framework proposed in this paper. From a broader methodological perspective, recent research has increasingly emphasized uncertainty-aware and reliability-driven learning under incomplete, heterogeneous, or dynamically evolving observations. For example, uncertainty quantification has been explored to improve the reliability of model inference under distributed and heterogeneous conditions (Chen et al. 2025). Acquisition-function design has also been studied as a principled way to balance information gain and decision cost in sequential query processes (Chen et al. 2026a). In addition, temporal adaptive modeling has been investigated for sparse and uncertain event sequences under evolving environments (Chen et al. 2026b). These studies provide useful conceptual references for understanding online learning as a process of uncertainty estimation, adaptive information acquisition, and reliability-oriented updating. Nevertheless, they are not specifically designed for multiclass imbalanced data streams with concept drift and limited labels. Compared with these broader uncertainty-aware learning paradigms, DMAE focuses on the streaming classification setting and integrates query-source separation, class-pair-level uncertainty control, drift-aware memory reconstruction, and weighted online ensemble updating into a unified active learning framework.

2.5 Methods

This section presents the proposed DMAE framework for multiclass imbalanced concept-drifting data streams. DMAE integrates four tightly coupled components: a hybrid uncertainty-based querying mechanism built on the decomposable asymmetric margin-threshold matrix and BAM^2-Lite, a drift-strength-aware dual-memory initialization scheme, a utility-driven soft replacement strategy for candidate classifiers, and a comprehensive sample-weighting formulation PCN-Weight. Together, these components enable unified coordination of label querying, memory maintenance, drift response, and incremental ensemble updating under limited labeling budgets. The remainder of this section is organized as follows. Section 3.1 presents the overall framework and online workflow of DMAE. Section 3.2 introduces the hybrid uncertainty and querying strategy. Section 3.3 describes the drift-strength-aware dual-memory initialization and soft replacement mechanism. Section 3.4 details the PCN-Weight formulation. Section 3.5 analyzes the time and space complexities of the proposed method.

2.6 Overall framework and workflow

This subsection presents the overall framework and online workflow of DMAE. DMAE integrates four tightly coupled components, namely hybrid uncertainty-based querying via BAM^2-Lite, drift-strength-aware dual-memory initialization via SDMI, utility-driven soft replacement via PDSR, and weighted online updating via PCN-Weight. These components are organized within a unified online loop to coordinate prediction, budgeted label querying, class-distribution estimation, memory maintenance, drift response, and ensemble evolution. In this way, DMAE does not treat active querying, imbalance estimation, drift adaptation, and ensemble replacement as isolated operations, but connects them through a continuous online learning process.

As illustrated in Fig. 1, the framework mainly consists of an online ensemble classifier E composed of D base classifiers, an ADWIN-based drift detection module, a label sliding window L for approximately unbiased estimation of the current class distribution, class-wise short-term sample windows R_c, long-term prototype memories H_c, and an initialization training sequence I used to reconstruct a candidate classifier after drift detection. For each incoming instance, the current ensemble first produces posterior probabilities and a margin-based confidence estimate. Under the global labeling budget, the hybrid querying module then decides whether to request the true label through random querying or BAM^2-Lite -guided active querying. Once a label is acquired, the sample is used for PCN-Weight-guided online updating and class-wise memory maintenance. Meanwhile, ADWIN monitors online prediction errors under the available queried labels and estimates the drift strength. When significant drift is detected, DMAE invokes SDMI to reconstruct a candidate classifier from the dual-memory structure, and PDSR then determines whether the candidate should replace an existing ensemble member after a trial period.

Let the data stream be denoted as $S = \{x_1, x_2, \dots, x_t, \dots\}$, where x_t is the incoming instance at time t, and its true label y_t is unavailable unless the algorithm explicitly requests it from an oracle. For each incoming instance, the ensemble classifier E outputs a posterior probability vector p_t, the predicted label $\widehat{y}_t$, and a margin score m_t. The posterior vector and margin are formally defined in Section 3.2. A smaller margin indicates that the ensemble has lower confidence in distinguishing the two most probable classes, and therefore the margin is used as the basic uncertainty signal for subsequent BAM^2-Lite -based active querying.

Under the global labeling budget, DMAE adopts a hybrid querying strategy that combines random querying and BAM^2-Lite -guided active querying. Let b denote the labeling-budget ratio, t denote the number of processed instances, and l denote the number of labels already queried from the oracle. At time t, the maximum number of labels

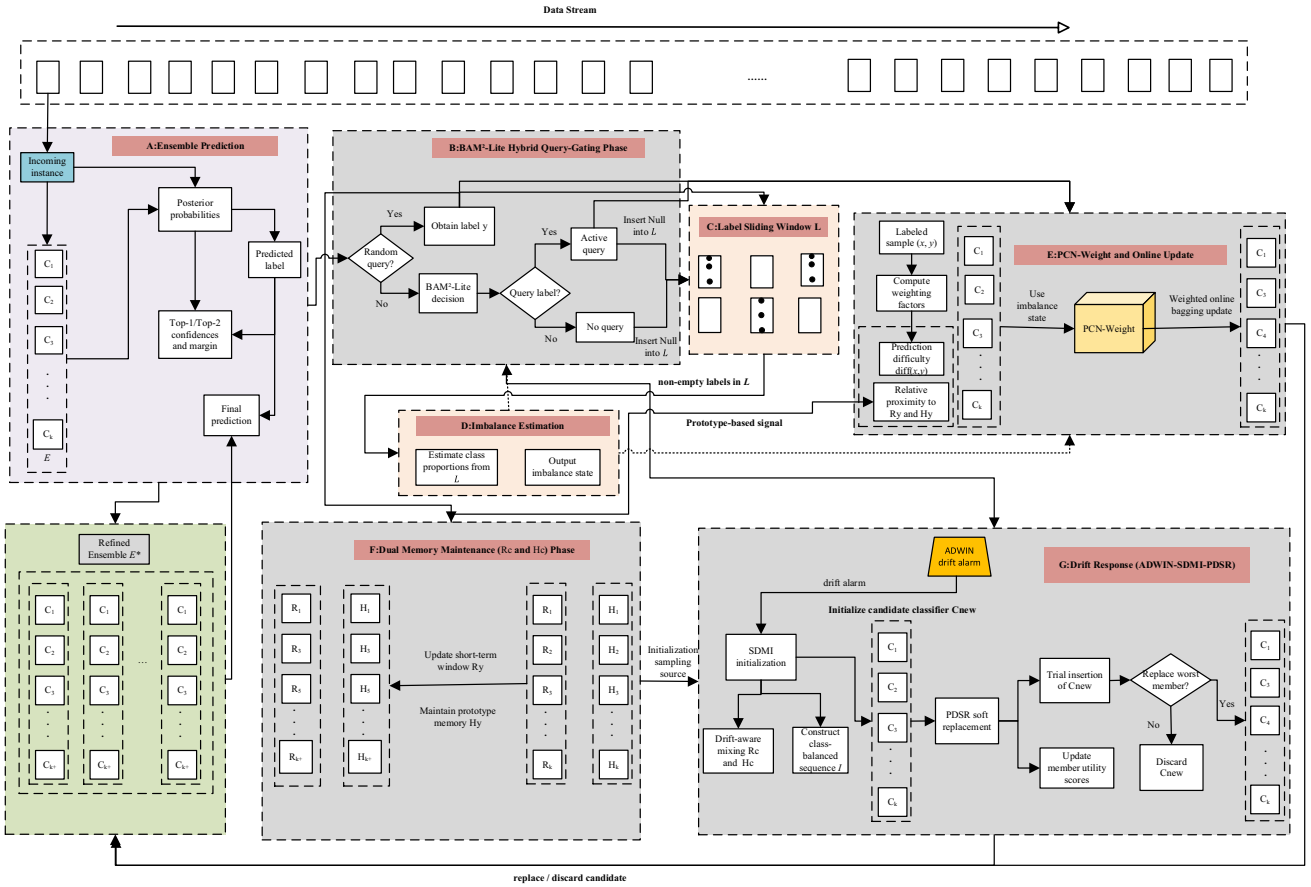


Fig. 1 Overall online workflow of DMAE

allowed by the cumulative budget is $B_t = \lfloor bt \rfloor$. A new label can be requested only when $l < B_t$. Thus, random querying, BAM^2-Lite uncertainty querying, selective sampling, and drift-stage exploration all share the same global labeling budget. If the remaining budget is exhausted, DMAE does not request the label of the current instance and pushes a Null placeholder into the label sliding window. SDMI and PDSR do not consume additional labels; they only reuse the labels already obtained under the same budget.

To clarify the label-querying workflow, DMAE distinguishes three cases for each incoming instance. First, if the random-query branch is triggered and the global budget is still available, the oracle label y_t is requested. This label is inserted into the label sliding window L, and the labeled sample (x_t, y_t) is used for model updating, memory maintenance, and drift monitoring. Second, if the instance is selected by the BAM^2-Lite active-query branch, the oracle label y_t is also requested and used for ensemble updating, short-term sample-window updating, long-term prototype-memory updating, BAM^2-Lite parameter adaptation, and PDSR utility evaluation. However, because actively queried instances are concentrated around uncertain decision boundaries, directly using them for class-distribution estimation may introduce query-selection bias. Therefore, the corresponding position in the label sliding window L is filled with Null rather than y_t. Third, if the instance is not queried, Null is pushed into L, and the instance does not participate in supervised model updating.

Accordingly, the label sliding window L is used mainly for approximately unbiased estimation of the current class distribution based on random-query labels. Let $n_c(t)$ denote the number of non-null labels of class c in the current label window. The estimated class prior can be computed as $\hat{\pi}_c(t) = n_c(t)/(\sum_{j=1}^{C} n_j(t) + \epsilon_0)$, where ϵ_0 is a small smoothing constant. Based on this estimated class distribution, DMAE obtains the current imbalance state of each class and provides class-distribution information for BAM^2-Lite and PCN-Weight. This design separates two different uses of queried labels: random-query labels provide a relatively unbiased statistical basis for class-distribution estimation, whereas active-query labels improve learning around informative and minority-relevant boundary regions.

Drift detection is also performed under the partial-label setting. Since unqueried instances have no ground-truth labels, ADWIN can only be updated when a queried label is available. To reduce the bias caused by boundary-focused

active querying, the prediction errors of random-query labels are used as the main error stream for ADWIN-based drift monitoring. For a random-query sample, the supervised error is computed as $e_t = \mathbb{I}(\hat{y}_t \neq y_t)$. Active-query labels are still used for model updating, memory updating, BAM^2-Lite adaptation, and PDSR utility evaluation. Their prediction errors may also be recorded as auxiliary evidence for estimating drift severity, but they are not used alone as the primary source for class-distribution estimation. This treatment avoids confusing the high uncertainty of actively queried boundary samples with true distributional drift.

Based on the above protocol, the online workflow of DMAE can be summarized as follows. For an incoming unlabeled instance x_t, the ensemble classifier first outputs p_t, $\hat{y}_t$, and m_t. The global budget condition $l < B_t$ is then checked. If the budget is available, the random-query branch may be triggered with probability ϵ. Otherwise, the BAM^2-Lite -based uncertainty strategy is invoked to determine whether the current instance should be queried according to the class-pair-level threshold and the margin m_t. If no query is made, Null is inserted into L, and the algorithm proceeds to the next instance.

When a label is requested, the sample is routed according to its query source. A random-query label is inserted into L and used for class-distribution estimation. An active-query label is used for training and memory updating, while its position in L is filled with Null to avoid contaminating the class-prior estimation. The non-null entries in L are then used to estimate the current class distribution and imbalance indicators. For each queried sample (x_t, y_t), DMAE computes its prediction difficulty, class-imbalance state, prototype-relative position, and temporal information, and obtains the comprehensive training weight through PCN-Weight. The weighted sample is used to update the ensemble classifier by online bagging. Meanwhile, it is stored in the corresponding short-term sample window R_{y_t}, and the long-term prototype memory H_{y_t} is updated according to the representativeness and diversity criteria. In this way, short-term memory captures recent local distributional information, whereas long-term memory preserves relatively stable representative patterns.

When ADWIN detects significant drift, DMAE computes the drift-strength indicator s_{drift} and invokes SDMI to construct an initialization sequence I from the short-term windows and long-term prototype memories. The mixing ratio between short-term and long-term memory is adaptively determined according to s_{drift}. A new candidate classifier C_{new} is then trained on I with time-decayed PCN weights. After SDMI initialization, C_{new} does not immediately replace an existing ensemble member. Instead, PDSR temporarily inserts it as a trial member and evaluates its utility during a limited trial period. The utility score jointly considers overall accuracy, minority-oriented performance, and prediction diversity. If C_{new} achieves better utility than at least one existing member, it replaces the member with the lowest utility; otherwise, it is discarded and the original ensemble structure remains unchanged. Through this trial-based mechanism, DMAE avoids abrupt replacement caused by short-term fluctuations or drift misdetection.

Figure 1 illustrates the complete online workflow of DMAE. For each incoming instance, the current ensemble first produces posterior probabilities and a margin-based confidence estimate. Under the global labeling budget, the hybrid querying module decides whether to request the true label through random querying or BAM^2-Lite -guided active querying. Queried labels are then routed differently according to their sources: random-query labels support approximately unbiased class-distribution estimation, while active-query labels support boundary-focused model updating and memory maintenance. When ADWIN detects significant drift, DMAE reconstructs a candidate classifier from the dual-memory structure through SDMI and performs trial-based soft replacement through PDSR.

Algorithm 1 presents the pseudocode of the complete DMAE learning procedure. At the beginning of learning, the algorithm initializes the ensemble classifier E, the label sliding window L, the class-wise short-term sample windows R_c, the long-term prototype memories H_c, the BAM^2-Lite state variables, the ADWIN detectors, and the PDSR trial state. For each incoming instance, Algorithm 1 first performs prediction and budget checking, and then determines whether to request the true label through random querying or BAM^2-Lite active querying. If the label is obtained, the sample is used for weighted ensemble updating, memory maintenance, BAM^2-Lite adaptation, and PDSR utility evaluation according to its query source. If drift is detected, the algorithm invokes SDMI to initialize a candidate classifier and then uses PDSR to decide whether the candidate should replace an existing member. In this way, Algorithm 1 unifies budgeted active querying, bias-controlled class-distribution estimation, PCN-Weight-based online updating, dual-memory-based drift response, and soft ensemble replacement within a single online framework.

Algorithm 1 DMAE-BAM^2-Lite-SDMI-PDSR Learning Procedure

Inputs:

1: S: data stream

2: D: number of base classifiers

3: C: number of classes

4: θ: base margin threshold

5: b: labelling budget $(0 < b \leq 1)$

6: ε: random sampling ratio

7: sizelab: size of the label sliding window

8: sizesam: size of each short-term sample window

9: T_{trial}: length of the PDSR trial period

Output: Ensemble classifier E

Initialization:

1: $t \leftarrow 0$ number of processed instances

2: $l \leftarrow 0$ number of labelled instances

3: Construct D base classifiers and form ensemble E

4: Create label window $L[1..sizelab]$and set all entries to Null

5: For each class $c = 1..C$:

6: Create short-term sample window $R_c[1..sizesam]$ and set empty

7: Create long-term memory set $H_c \leftarrow \emptyset$

8: Initialise BAM² state: $\alpha[1..C] \leftarrow 0$, $G[1..C][1..C] \leftarrow 0$, drift-severity EWMA $s_{\text{drift}} \leftarrow 0$

9: Initialise ADWIN detectors for each base classifier in E

10: trialClassifier ←Null; trialCounter ← 0; reset all PDSR statistics

Process:

```
1: while S.hasNext() do
2:    x ← S.nextInstance(); t ← t + 1
3:    labelling ← False; querySource ← Null; corridorFlag ← False; y ← Null
4:    B_t = ⌊bt⌋
5:    if t ≤ sizelab then
6:       y ← queryOracle(x)
7:       labelling ← True; querySource ← Random
8:       Push y into label window L
9:    else if  l < B_t then
10:       if Uniform(0,1) ≤ ε then
11:          y ← queryOracle(x)
12:          labelling ← True; querySource ← Random
13:          Push y into label window L
14:       else
```

```
        (labelling, corridorFlag) ← UncertaintySelectiveStrategy_BAM²Lite(E, x, θ, b, t, l, L, α, G, s_drift)
        if labelling = True then
            y ← queryOracle(x)
            querySource ← Active
            Push Null into label window L
        else
            Push Null into label window L
        end if
    end if
  else
    Push Null into label window L
  end if
  if labelling = True then
    l ← l + 1
    (diff, imb_y, weight) ← ComputeSampleWeight(E, x, y, L)
    UpdateEnsemble(E, x, y, weight)
    UpdateShortTermBuffer(R_y, x, y, t, weight)
    UpdateLongTermMemory(H_y, x, y, weight)
    if corridorFlag = True then
      UpdateBAM²ParamsLite(α, G, y, x)
    end if
    UpdatePDSRStatistics(E, trialClassifier, x, y)
  end if
  if querySource = Random then
    UpdateADWINAndDriftEWMA(E, x, y, s_drift)
  end if
  if AnyADWINDetectsDrift() then
    ρ ← ComputeMixRatio(s_drift)
    C_new ← CreateBaseClassifier()
    I ← BuildInitSequence_SDMI(R, H, ρ)
    TrainWithTimeDecay(C_new, I)
    StartPDSRTrial(E, C_new, T_trial)
  end if
  if PDSRTrialFinished() then
    k* ← ArgMinUtility(E)
    UpdateEnsembleByPDSR(E, k*)
  end if
end while
```

For each incoming instance, the current ensemble first produces posterior probabilities and a margin-based confidence estimate. Under the global labeling budget, the hybrid querying module decides whether to request the true label through random querying or BAM^2-Lite -guided uncertainty querying. Different queried labels are then routed according to their sources. Random-query labels are inserted into the label sliding window L for approximately unbiased class-distribution estimation, whereas BAM^2-Lite active-query labels are used for PCN-Weight-guided online updating and class-wise memory maintenance but are recorded as Null in L. When ADWIN detects significant drift from the random-query error stream,

DMAE reconstructs a candidate classifier from the dual-memory structure and performs trial-based soft replacement through PDSR.

Algorithm 1 presents the pseudocode of the DMAE learning procedure. Let S be an infinite data stream containing unlabeled instances. At the beginning of the learning stage, the algorithm constructs an ensemble E consisting of D base classifiers and initializes the label sliding window L, the short-term sample window R_c, and the long-term memory set H_c for each class. It also initializes the potential vector α of BAM^2-Lite, the interaction matrix G, the drift intensity indicator s_{drift}, the ADWIN detectors, and the PDSR trial state. The label sliding window is used to estimate the current multiclass imbalance state online; the short-term window characterizes the local distribution over the most recent period; and the long-term memory captures relatively stable class patterns through prototype samples.

For each unlabeled instance x arriving in the data stream, Algorithm 1 first checks whether the instance belongs to the initial label-window filling stage. These initial random labels are pushed into L and counted in the total number of queried labels. After this stage, the algorithm enforces the cumulative labeling budget. At time t, the maximum number of labels allowed by the current budget is $B_t = \lfloor bt \rfloor$, and a new label can be requested only when $l < B_t$. If the budget is available, the algorithm first performs random querying with probability ε. If random querying is not triggered, BAM^2-Lite is invoked to determine whether the current sample should be actively queried according to the margin, the class-pair threshold, the remaining budget, and the drift intensity. If the budget is exhausted or the current sample is not selected, Null is pushed into L.

Once the algorithm requests the true label y, the labeled sample (x, y) is used for model updating and memory maintenance. However, its entry in the label sliding window depends on the query source. If the label is obtained from the random-query branch, y is pushed into L and contributes to class-distribution estimation. If the label is obtained from the BAM^2-Lite active-query branch, the sample is used for ensemble updating, short-term sample-window updating, long-term prototype-memory updating, BAM^2-Lite parameter adaptation, and PDSR utility evaluation, but Null is pushed into L. This design prevents actively queried boundary samples from biasing the class-prior estimation in the label sliding window.

For each queried sample (x, y), DMAE computes the classification difficulty $diff(x, y)$, the class-imbalance indicator imb_y, and the training weight $weight(x, y)$ according to PCN-Weight. The weighted sample is then used to update the ensemble E through online bagging. Meanwhile, the sample is written into the short-term sample window R_y of its corresponding class, and the long-term memory H_y is updated according to representativeness and diversity criteria. If the current queried sample lies within the BAM^2-Lite threshold corridor, its feedback is further used to update the potential vector α and interaction matrix G, so that the class-pair margin thresholds can gradually adapt to the evolving imbalance and drift structure.

To respond to concept drift under partial labeling, Algorithm 1 updates ADWIN mainly using prediction errors from random-query labels. For a random-query sample, the supervised error is computed as $e_t = \mathbb{I}(\hat{y}_t \neq y_t)$. This avoids using only boundary-focused active-query samples as the main drift signal, thereby reducing the risk of confusing local uncertainty with true distributional drift. When any ADWIN detector reports significant drift, the algorithm computes the current drift intensity s_{drift} and obtains the mixing ratio $\rho(s_{\text{drift}})$ between short-term and long-term memories. It then creates a new base classifier C_{new}, draws samples from each class's short-term window R_c and long-term memory H_c according to ρ, constructs a class-balanced initialization training sequence I, and performs offline initialization training for C_{new} under time-decay weights, which corresponds to the SDMI mechanism.

After initialization, C_{new} does not immediately hard-replace any existing model. Instead, it temporarily joins the ensemble and enters a PDSR trial period of length T_{trial}. During the trial period, the algorithm maintains a utility score for each base classifier on labeled samples, jointly considering local accuracy, minority-class performance, and prediction diversity. At the end of the trial, the member with the lowest utility is selected and denoted as C_{k^*}. If C_{k^*} corresponds to an old model, it is replaced by C_{new}; if C_{k^*} is the trial model itself, C_{new} is discarded and the ensemble structure remains unchanged. Through this procedure, Algorithm 1 unifies budgeted label querying, bias-controlled label-window updating, PCN-Weight-based sample weighting, dual-memory-based drift response, and PDSR-based soft replacement within a single online framework.

2.7 Hybrid uncertainty and querying strategy based on BAM^2-lite

This subsection presents the hybrid uncertainty and querying mechanism of DMAE for multiclass imbalanced data streams. Centered on a decomposable asymmetric margin-threshold structure and its corridor-gated BAM^2-Lite strategy, the proposed mechanism unifies random querying, structured uncertainty assessment, and selective sampling within a single framework. The random-query branch provides the label sliding window with an approximately unbiased estimate of the class distribution, whereas BAM^2-Lite focuses on selecting informative boundary samples for model updating and memory maintenance. The decomposable asymmetric margin-threshold structure characterizes, at the class-pair scale, the separability among different classes and the directional querying preference. BAM^2-Lite further

introduces class potential, pairwise interaction, drift intensity, and imbalance-related information to parameterize and smoothly control these pairwise thresholds, thereby prioritizing the acquisition of valuable boundary samples under a strict labeling budget.

Let the ensemble classifier be E, and denote the predicted posterior probability of an incoming stream instance x as $p_E(c \mid x)$. Then y_{c_1} is defined as in Eq. (1).

$$y_{c_1} = \arg\max_c p_E(c \mid x), \quad y_{c_2} = \arg\max_{c \neq y_{c_1}} p_E(c \mid x) \tag{1}$$

The margin is defined as in Eq. (2).

$$\text{margin}(x) = p_E(y_{c_1} \mid x) - p_E(y_{c_2} \mid x) \tag{2}$$

To explicitly distinguish directional heterogeneity among classes during active querying, DMAE maintains class-pair-level threshold information for each ordered class pair (i, j). Unlike a fixed global threshold, the class-pair threshold is adjusted according to the evolving class distribution and the historical confusion pattern between classes. In the proposed implementation, the label sliding window L provides an online estimate of the current imbalance state based on its non-null random-query labels. Since BAM^2-Lite is used before the true label of the current active-query sample is obtained, Algorithm 2 does not directly update the label window or the class-pair state. Instead, it only determines whether the current sample should be actively queried and whether it lies in the uncertainty corridor. After Algorithm 1 obtains the true label of a queried corridor sample, the BAM^2-Lite state variables are updated through UpdateBAM^{2}ParamsLite. This design separates active-query decision making from label acquisition and label-window maintenance. The potential vector α records class-level learning difficulty or querying tendency, the interaction matrix G records directional confusion between class pairs, and the imbalance-related term reflects the current class-distribution bias estimated from L. Together, these components gradually adjust the initially uniform threshold behaviour into a direction-sensitive class-pair threshold field, while keeping the global label-routing rule consistent with Algorithm 1.

Based on the above class-pair threshold information, the basic uncertainty strategy can be described as follows: if $\text{margin}(x) \leq M_{\text{eff}}$, the ensemble is considered unable to confidently distinguish y_{c_1} from y_{c_2}, and the current instance is selected for active querying. Otherwise, the algorithm enters the selective sampling stage and decides whether to further query this sample by considering the remaining labeling budget and the redundancy margin $\text{margin}(x) - M_{\text{eff}}$. This design ensures sufficient attention to hard samples near the decision boundary and avoids over-consuming the limited budget on highly confident samples. To further improve adaptability and interpretability, BAM^2-Lite constructs a decomposable asymmetric margin-threshold component for each ordered class pair (i, j), as shown in Eq. (3).

$$\widetilde{M}_{ij} = \theta\, \sigma(\alpha_i - \alpha_j) + \lambda_G\, G_{ij} + \mu_{drift}\, s_{drift} + \nu_{imb}\, \phi_{ij} \tag{3}$$

In Eq. (3), each term has a specific practical meaning. The potential parameter α_i describes the class-level querying tendency of class i. A larger difference $\alpha_i - \alpha_j$ indicates that the ordered class pair (i, j) requires a different query sensitivity from its reverse direction (j, i). Therefore, the term $\sigma(\alpha_i - \alpha_j)$ introduces directional asymmetry into the threshold component. The interaction term G_{ij} records the historical confusion tendency from class i to class j. If samples predicted as class i are frequently confused with class j, G_{ij} increases the influence of this class pair in the active-query decision. The drift term s_{drift} represents the current concept-drift intensity estimated from the ADWIN-based error changes. When the stream becomes unstable, this term increases the sensitivity of BAM^2-Lite to boundary samples in the drift period. The imbalance-related term ϕ_{ij} reflects the current class-pair imbalance structure estimated from the non-null entries in the label sliding window L. It can be constructed from the relative imbalance indicators of classes i and j, for example through the conditional frequency ratio $\log \frac{1+\text{imb}_i}{1+\text{imb}_j}$. The coefficients λ_G, μ_{drift}, and ν_{imb} control the contributions of historical confusion, drift intensity, and imbalance bias, respectively. Therefore, $\widetilde{M}_{ij}$ is not a single heuristic threshold, but a decomposable class-pair component that combines four interpretable factors: class-level potential difference, pairwise confusion, drift intensity, and imbalance structure. This design allows BAM^2-Lite to distinguish different ordered class pairs rather than applying the same threshold to all class boundaries. It also explains why the threshold structure is asymmetric: the query preference from class i to class j may be different from that from class j to class i, especially when one class is underrepresented or when the two directions show different historical confusion patterns. In this way, the decomposable structure provides an interpretable basis for adaptive class-pair-level uncertainty querying.

On top of the decomposable structure, we introduce BAM^2-Lite to smoothly control the threshold matrix and enable it progressively. First, we linearly interpolate the recommended threshold $\widetilde{M}_{ij}$ with the global baseline threshold θ using a dynamic mixing coefficient $\beta_{eff} \in [0,1]$, yielding the BAM^2-Lite class-pair threshold as in Eq. (4).

$$M_{ij}^{BAM2} = (1 - \beta_{eff})\, \theta + \beta_{eff}\, \widetilde{M}_{ij} \tag{4}$$

The mixing coefficient β_{eff} is adaptively adjusted according to the consumed labeling-budget ratio, the current drift intensity, and the predictive uncertainty of the sample. The final β_{eff} is defined as in Eq. (5).

$$\beta_{eff} = \mathrm{clip}_{[0,1]}(\beta_{min} + (\beta_{max} - \beta_{min})\, r_{label} + \kappa_1\, s_{drift} + \kappa_2\,(1 - p_{max})) \tag{5}$$

where r_{label} is the ratio of consumed labeling budget, $p_{max} = \max_c p_E(c \mid x)$ is the maximum posterior probability of the current sample, and $\beta_{min}, \beta_{max}, \kappa_1, \kappa_2$ are hyperparameters; $\mathrm{clip}_{[0,1]}(\cdot)$ truncates the result into [0,1]. Thus, when labeling history is limited, drift is not evident, or the prediction is highly confident, β_{eff} stays close to β_{min} and the model mainly follows the original threshold; as the budget becomes more available, drift intensifies, and predictions become more uncertain, β_{eff} increases smoothly, and the structured information (class potential, confusion, imbalance, etc.) gradually exerts stronger influence on class-pair thresholds.

To maintain overall stability while introducing structured thresholds, BAM^2-Lite further adopts a corridor mechanism together with a conservative threshold cap. Let the margin of the current sample be $\mathrm{margin}(x)$. When its distance to the baseline threshold θ satisfies $|\,\mathrm{margin}(x) - \theta\,| \leq \delta$, the sample is regarded as falling inside the uncertainty corridor. Only in this case does BAM^2-Lite activate the decomposable class-pair threshold adjustment. Outside the corridor, the strategy falls back to the baseline threshold, so as to avoid unnecessary structured perturbation in high-confidence regions. Inside the corridor, the final uncertainty threshold used for querying is defined as in Eq. (6).

$$M_{eff}(y_{c_1}, y_{c_2}) = \min(\theta,\ M^{BAM2}_{y_{c_1} y_{c_2}}) \tag{6}$$

Since the query decision is triggered when $\mathrm{margin}(x) \leq M_{\mathrm{eff}}(y_{c_1}, y_{c_2})$, Eq. (6) ensures that the adaptive threshold does not exceed the global baseline threshold θ. Therefore, BAM^2 -Lite does not arbitrarily enlarge the query region beyond the baseline uncertainty range. This conservative cap is important under a strict labelling budget because it prevents the structured class-pair adjustment from over-consuming labels, while still allowing the model to refine the query decision for samples located in the sensitive threshold corridor.

In summary, the proposed hybrid uncertainty and querying strategy is jointly driven by three layers of mechanisms. First, random querying provides an approximately unbiased statistical foundation for estimating class imbalance. Second, the decomposable asymmetric margin-threshold structure explicitly characterizes direction-sensitive querying preference at the class-pair scale by encoding class potential, confusion patterns, drift intensity, and imbalance structure into the threshold space. Third, BAM^2-Lite performs fine-grained threshold refinement for key boundary samples inside the uncertainty corridor through dynamic mixing and the conservative threshold cap. Their combination enables the algorithm, under strict labelling-budget constraints, to prioritize informative boundary samples around minority classes and drift intervals, while avoiding excessive label consumption in high-confidence regions.

Algorithm 2 UncertaintySelectiveStrategy_BAM2Lite *(E,x,θ,b,t,l,L,a,G,s_{drift}*

Inputs:

1: E: ensemble classifier

2: x: current unlabelled instance

3: θ: base margin threshold

4: b: labelling budget $(0 < b \leq 1)$

5: t: number of processed instances

6: l: number of labelled instances

7: L: label sliding window

8: α[1..C], G[1..C][1..C]: BAM²-Lite state

9: s_{drift}: current drift-severity EWMA

Output: A Boolean variable labelling ∈ {True, False}, and a Boolean variable corridorFlag ∈ {True, False}

Process:

1: Make predictions on x by E, obtain posterior probabilities $p_E(c \mid x)$

2: Let y_{c_1} be the class with maximum probability, and y_{c_2} be the second one

3: Compute $\text{margin}(x) = p_E(y_{c_1} \mid x) - p_E(y_{c_2} \mid x)$

4: Compute current labelling-cost ratio: costNow ← $l/\max(t,1)$

5: labelling ← False; corridorFlag ← False

6: // Step 1: BAM²-Lite corridor & effective threshold

7: if $|\text{margin}(x) - \theta| \leq \delta$ then

8: corridorFlag ← True

9: Compute $\widetilde{M}_{y_{c_1},y_{c_2}} = \theta \cdot \sigma\left(\alpha_{y_{c_1}} - \alpha_{y_{c_2}}\right) + \lambda_G G_{y_{c_1},y_{c_2}} + \mu_{\text{drift}} s_{\text{drift}} + \nu_{\text{imb}} \phi_{y_{c_1},y_{c_2}}$

10: Compute dynamic mixing weight $\beta_{\text{eff}} \in [0,1]$ using costNow, s_{drift} and $p_E(y_{c_1} \mid x)$

11: $M^{\text{BAM2}}_{y_{c_1},y_{c_2}} \leftarrow (1-\beta_{\text{eff}})\theta + \beta_{\text{eff}} \widetilde{M}_{y_{c_1},y_{c_2}}$

12: $M_{\text{eff}} \leftarrow \min\left(\theta, M^{\text{BAM2}}_{y_{c_1},y_{c_2}}\right)$

13: else

14: $M_{\text{eff}} \leftarrow 0$

15: end if

16: // Step 2: uncertainty-based decision

17: if $\text{margin}(x) \leq M_{\text{eff}}$ then

18: labelling ← True

19: return (labelling, corridorFlag)

20: end if

21: // Step 3: selective sampling under remaining budget

22: sampbudget ← b − costNow

23: if sampbudget > 0 then

24: $q \leftarrow \text{margin}(x) - M_{\text{eff}}$

25: sampbudget ← sampbudget / (sampbudget + max{q,0})

26: Generate a uniform random variable $\zeta \sim \text{Uniform}(0,1)$

27: if $\zeta \leq$ sampbudget then

28: labelling ← True

29: end if

30: end if

31: return (labelling, corridorFlag)

Algorithm 2 provides the pseudocode implementation of the hybrid uncertainty and selective-labelling strategy in the overall DMAE framework. The inputs include the ensemble classifier E, the current unlabelled instance x, the global baseline threshold θ, the labelling budget b, the number of processed samples t, the number of labelled samples l, the label sliding window L, the BAM^2-Lite state variables α and G, and the drift-intensity index s_{drift}. The outputs are two Boolean variables, namely labelling and corridorFlag. The variable labelling indicates whether the current instance is selected for active labelling by BAM^2-Lite, while corridorFlag indicates whether the instance falls into the uncertainty corridor and can be used for subsequent BAM^2-Lite state updating after its true label is obtained in Algorithm 1.

Lines 1–5 complete the basic measurements of the current instance. The ensemble classifier E outputs the posterior distribution $p_E(c \mid x)$, determines the top-1 and top-2 predicted classes y_{c_1} and y_{c_2}, and computes the margin as $\text{margin}(x) = p_E(y_{c_1} \mid x) - p_E(y_{c_2} \mid x)$. Meanwhile, the current labelling-cost ratio is computed by $\text{costNow} = \frac{l}{\max(t,1)}$. Then, labelling and corridorFlag are initialized to False for the subsequent decision process. Lines 6–15 correspond to the corridor-based BAM^2-Lite threshold adjustment. The algorithm first checks whether the current sample lies near the global baseline threshold θ. If $|\text{margin}(x) - \theta| \leq \delta$, the sample is regarded as falling into the uncertainty corridor, and corridorFlag is set to True. In this case, BAM^2-Lite computes a decomposed class-pair threshold component $\widetilde{M}_{y_{c_1}, y_{c_2}}$ according to the class-potential difference, class-pair interaction, drift intensity, and imbalance-related term. Then, the dynamic mixing coefficient β_{eff} is used to combine the global baseline threshold with the decomposed class-pair threshold, yielding $M^{\text{BAM}^2}_{y_{c_1}, y_{c_2}}$. Finally, the effective threshold is obtained as $M_{\text{eff}} = \min\left(\theta, M^{\text{BAM}^2}_{y_{c_1}, y_{c_2}}\right)$. This corridor mechanism restricts structured threshold adjustment to the region where the query decision is sensitive, thereby avoiding unnecessary perturbation in high-confidence regions. If the sample is outside the corridor, the strategy falls back to the global baseline threshold by setting $M_{\text{eff}} = \theta$. Lines 17–20 implement the uncertainty-based active-query decision. When $\text{margin}(x) \leq M_{\text{eff}}$, the ensemble classifier is considered insufficiently confident in distinguishing y_{c_1} from y_{c_2}. Therefore, the current instance is regarded as lying near a critical decision boundary, and labelling is set to True. In this case, Algorithm 2 immediately returns the decision signals to Algorithm 1. The actual oracle query is not performed in Algorithm 2; instead, Algorithm 1 uniformly requests the true label and routes the queried label according to its source. Lines 21–30 correspond to the selective sampling stage under the remaining budget. For instances whose margin is slightly larger than M_{eff}, relying only on the strict uncertainty condition may be overly conservative. Therefore, Algorithm 2 further considers the remaining budget and the redundancy distance from the threshold. It first computes the remaining sampling budget as $\text{sampbudget} = b - \text{costNow}$, and the margin redundancy as $q = \text{margin}(x) - M_{\text{eff}}$. When $\text{sampbudget} > 0$, the supplementary sampling probability is adaptively computed by $\text{sampbudget} \leftarrow \frac{\text{sampbudget}}{\text{sampbudget}+\max(q,0)}$. Then, a random variable $\zeta \sim \text{Uniform}(0,1)$ is generated. If $\zeta \leq \text{sampbudget}$, the current instance is also selected for labelling. This design allows the strategy to sample more near-boundary instances when the remaining budget is sufficient, while automatically reducing supplementary sampling when the budget is tight.

It should be noted that Algorithm 2 only returns the decision signals labelling and corridorFlag. It does not obtain the true label from the oracle and does not update the label sliding window L. The actual oracle query, the insertion of y or Null into L, and the subsequent model and memory updates are uniformly handled in Algorithm 1 according to the query source. In addition, Algorithm 2 does not directly update α and G. When Algorithm 2 returns corridorFlag = True and Algorithm 1 subsequently obtains the true label, the BAM^2-Lite state variables are updated through UpdateBAM^{2}ParamsLite. This design keeps the uncertainty-selection module independent from label acquisition and label-window maintenance, and ensures that all active-query labels are routed consistently under the global labelling budget.

2.8 Drift-strength-aware dual-memory initialization and soft replacement

This subsection presents SDMI-PDSR, the drift-response module of DMAE. SDMI-PDSR combines drift-strength-aware dual-memory initialization with a soft replacement mechanism for candidate classifiers. For each class, DMAE maintains a short-term sample window and a long-term prototype memory, and uses the online-estimated drift strength to adaptively determine their mixing ratio during candidate-model initialization. The newly initialized classifier is then evaluated through a utility criterion before retention and ensemble replacement, so that memory reconstruction, initialization training, and member updating are coordinated within a structured and controllable procedure.

To leverage both the latest distribution and long-term patterns during initialization, DMAE maintains a two-level memory structure for each class c. The first level is a short-term sample window R_c with capacity sizesam, which mainly stores representative samples that have been updated under PCN-Weight in a recent period, used to characterize the current distribution and its local decision boundary. The second level is a long-term memory set H_c, which maintains a small number of relatively drift-stable prototype samples via simple representativeness and diversity criteria, used to describe the overall geometry of the class and the long-term contour of minority classes. During online operation, whenever a new labeled sample (x, y, t) arrives, it is first written into R_y, and H_y is updated when certain representativeness or novelty conditions are satisfied, gradually forming a complementary short-term/long-term dual-memory structure. Short-term memory is highly sensitive to new concepts, whereas long-term memory provides structural constraints from old patterns.

When any ADWIN detector reports significant drift, a drift-intensity index $s_{drift} \in [0,1]$ is constructed using the mean increment of its error-rate estimates, and a piecewise linear mapping is used to adaptively determine the proportion of short-term memory used in the initialization samples, $\rho(s_{drift})$, as in Eq. (7).

$$\rho(s_{drift}) = \begin{cases} \rho_{min}, & s_{drift} \le \tau_{min} \\ \rho_{min} + \frac{s_{drift}-\tau_{min}}{\tau_{max}-\tau_{min}}(\rho_{max}-\rho_{min}), & \tau_{min} < s_{drift} < \tau_{max} \\ \rho_{max}, & s_{drift} \ge \tau_{max} \end{cases} \tag{7}$$

where $0 < \rho_{min} \le \rho_{max} \le 1$, and $\tau_{min} < \tau_{max}$ control the boundaries between mild, moderate, and severe drift. Intuitively, when s_{drift} approaches τ_{min}, $\rho(s_{drift})$ approaches the lower bound ρ_{min} and initialization relies more on long-term memory H_c to preserve historical stable patterns; when s_{drift} approaches τ_{max}, $\rho(s_{drift})$ moves toward the upper bound ρ_{max} and initialization relies more on the short-term window R_c to quickly adapt to the new distribution; under moderate drift, $\rho(s_{drift})$ transitions smoothly between the two, providing a tunable trade-off between retention and forgetting.

Given the mixing ratio $\rho(s_{drift})$, DMAE samples from the short-term window and long-term memory for each class $c \in \{1, \ldots, C\}$ to construct a class-balanced initialization candidate set. Let the current number of valid samples in the short-term window be $|R_c|$, and let sizesam be the per-class target capacity. Then the number of samples drawn without replacement from R_c is $n_c^{(R)}$, defined in Eq. (8).

$$n_c^{(R)} = \lfloor \rho(s_{drift}) \cdot \min(|R_c|, \text{sizesam}) \rfloor \tag{8}$$

To keep the per-class sample size approximately consistent, the number of samples drawn from the long-term memory H_c is $n_c^{(H)}$, defined in Eq. (9).

$$n_c^{(H)} = \min(|R_c|, \text{sizesam}) - n_c^{(R)} \tag{9}$$

If H_c contains fewer than $n_c^{(H)}$ samples, all samples in H_c are selected. Merging the two parts yields the class-specific initialization candidate set I_c as in Eq. (10).

$$I_c = I_c^{short} \cup I_c^{long} \tag{10}$$

Then, concatenating candidates across all classes and sorting by arrival time t_i in ascending order yields the global initialization training sequence I as in Eq. (11).

$$I = \text{SortByTime}\left(\bigcup_{c=1}^{C} I_c\right) \tag{11}$$

where each element is recorded as $(x_i, y_i, t_i, \text{diff}_i)$, including the sample features, true class, arrival time, and the comprehensive classification difficulty diff_i. With this construction, the initialization sequence is approximately balanced across classes, preserves causal order in time, and explicitly carries sample-difficulty information.

After obtaining the time-ordered initialization sequence I, we introduce a time-decay mechanism on top of the comprehensive sample weights. Let $w(x_i, y_i)$ denote the comprehensive weight computed from classification difficulty and class imbalance ratio (PCN-Weight). For each initialization sample, we introduce an exponentially decaying factor with time as in Eq. (12).

$$\eta_i = \exp(-\gamma \frac{t_{\text{now}} - t_i}{\tau}) \tag{12}$$

where t_{now} is the drift time, and $\gamma, \tau > 0$ are hyperparameters controlling the decay slope and time scale, respectively. The new weight $\widetilde{w}_i$ is defined in Eq. (13).

$$\widetilde{w}_i = w(x_i, y_i) \cdot \eta_i \tag{13}$$

This design allows the initialization stage to still leverage historical minority samples to stabilize the decision boundary, while automatically emphasizing recent samples, in particular recent hard minority samples, thereby providing a controllable trade-off between memory retention and rapid adaptation.

Next, the algorithm adopts an online-bagging style initialization training for the new base classifier C_{new} over sequence I. For each sample $(\mathrm{x}_i, y_i, \widetilde{w}_i)$, a repetition count is generated from a Poisson distribution $k_i \sim \mathrm{Poisson}(\widetilde{w}_i)$, and the replicated samples are used to perform several incremental updates on C_{new}. Consequently, the new base classifier is initialized under three constraints, namely class balance, difficulty awareness, and time decay, so that from the beginning it can jointly accommodate recent distribution adaptation and long-term pattern inheritance, thereby substantially alleviating the cold-start issue induced by drift response. At this point, SDMI completes the dual-memory-based initialization process conditioned on the drift intensity.

Although SDMI significantly improves initialization quality, two risks remain if the conventional hard replacement rule is still applied, that is, directly replacing the existing member with the largest ADWIN error by C_{new}. First, the generalization ability of the new model under the new distribution has not been sufficiently validated; good performance on the initialization samples may not persist on subsequent data. Second, the replaced old model may still provide irreplaceable contributions in minority regions or in ensemble diversity; eliminating members purely by overall error can mistakenly remove components that are crucial to robustness. Therefore, DMAE assigns a finite trial period to the new model and compares new and old members using a utility score driven by performance and diversity.

Specifically, suppose the current ensemble contains D base classifiers $\{C_k\}_{k=1}^{D}$. Over a recent labeled window, a comprehensive utility score U_k is maintained for each base classifier as in Eq. (14).

$$U_k = \lambda_{acc}(1 - \widehat{e}^{\,k}) + \lambda_{imb}\,\widehat{GMean}^{\,k} + \lambda_{div}\,\widehat{\mathrm{Div}}^{\,k} \tag{14}$$

where $\widehat{e}^{\,k}$ is the error rate on the k sliding window characterizing overall accuracy; $\widehat{GMean}^{\,k}$ is the minority-oriented G-Mean on the same window measuring adaptation to class imbalance; $\widehat{\mathrm{Div}}^{\,k}$ denotes the disagreement degree between this member and other members' predictions (larger indicates higher diversity); and $\lambda_{acc}, \lambda_{imb}, \lambda_{div} \geq 0$ are weights controlling the relative importance of the three terms.

After SDMI training, C_{new} does not immediately replace any old member. Instead, it is softly inserted into the ensemble, forming a temporary ensemble of size $D+1$, participates in prediction with a small initial weight, and initializes its utility score $U_{new} = 0$ along with a trial counter. Then, during a trial period of length T_{trial}, all newly arriving labeled samples are used both for routine model updates and for utility-statistics updates; on a unified sliding window, each classifier's $\widehat{e}^{\,k}$, $\widehat{GMean}^{\,k}$, $\widehat{\mathrm{Div}}^{\,k}$, and U_k are updated synchronously. This design allows each label to support BAM^2-Lite threshold adjustment and SDMI memory maintenance, while also providing evidence for PDSR utility evaluation, thereby maximizing the use of label information under a fixed labeling budget.

When the trial period ends, PDSR finds the index with the smallest utility score among all $D+1$ members: $k^* = \arg\min_k U_k$. If $k^* \neq$ new, then the candidate model C_{new} is better than at least one existing member under the joint measure of overall accuracy, minority performance, and diversity, and C_{new} replaces C_{k^*}, restoring the ensemble size to D. If $k^* =$ new, the candidate model fails to provide sufficient gain and is discarded, keeping the original ensemble unchanged. With the combined SDMI–PDSR mechanism, a new base classifier must pass a statistical trial screening that simultaneously considers overall accuracy, minority performance, and diversity before becoming a formal ensemble member, thereby significantly reducing the risk of catastrophic reset caused by drift mis-detection or short-term oscillations.

SDMI-PDSR is not independent of the uncertainty and querying strategy described earlier; it shares the same drift-intensity index s_{drift}, the same label sliding window, and the same dual-level sample memory structure with BAM^2-Lite. The uncertainty strategy uses s_{drift} to adaptively adjust class-pair thresholds; SDMI uses the same index to balance inheritance and forgetting between short-term samples and long-term prototypes; PDSR determines which base classifiers to retain under a unified utility scale. Together, the three components form a closed loop that answers, respectively, which samples to query, which memories to use for initializing new models, and which base classifiers to keep, providing a stable and efficient online response mechanism to mixed concept drift in multiclass imbalanced data streams.

Algorithm 3 SDMI-based InitialiseCandidate

Inputs:

E: current ensemble with D base classifiers $\{C_1, \ldots, C_D\}$

R_c $(c = 1, \ldots, C)$: short-term sample windows for each class

H_c $(c = 1, \ldots, C)$: long-term prototype memories for each class

s_{drift}: current drift severity (EWMA in [0,1])

sizesam: target size of the per-class initialisation set

$w(\cdot)$: comprehensive sample-weight function

$\rho_{\min}, \rho_{\max}, \gamma, \tau$: mixing and time-decay parameters

Output: Initialised candidate base classifier C_{new}.

Process (SDMI: severity-aware dual-memory initialisation):

Compute mixing ratio $\rho \leftarrow \rho(s_{\text{drift}}), \rho_{\min} \le \rho \le \rho_{\max}$

Set global initialisation sequence $I \leftarrow \emptyset$

For each class $c = 1, \ldots, C$ do

    Compute the number of samples drawn from short- and long-term memory

$n_c^{(R)} \leftarrow \lfloor \rho \cdot \min(\text{sizesam}, \mid R_c \mid) \rfloor$,

$$n_c^{(H)} \leftarrow \max\{0, \text{sizesam} - n_c^{(R)}\}$$

    Sample $n_c^{(R)}$ instances without replacement from R_c.

    Sample $n_c^{(H)}$ instances without replacement from H_c.

    Insert all selected samples of class c into I, keeping their arrival time t_i and stored difficulty value diff_i.

End for

Sort all samples in I by their arrival time t_i in ascending order.

Create a new base classifier C_{new} and reset its parameters.

For each sample $(x_i, y_i, t_i, \text{diff}_i) \in I$ do

    Compute temporal decay factor $\eta_i \leftarrow \exp(-\gamma \frac{t_{\text{now}} - t_i}{\tau})$.

    Compute base weight (PCN-Weight)

$w_i^{\text{base}} \leftarrow w(x_i, y_i, \text{diff}_i)$.

    Set effective training weight

$\widetilde{w}_i \leftarrow w_i^{\text{base}} \cdot \eta_i$.

    Draw an integer multiplicity

$k \sim \text{Poisson}(\widetilde{w}_i)$.

    If $k > 0$ then train C_{new} on (x_i, y_i) with weight k.

End for

Return C_{new}

Algorithm 3 presents the pseudocode of the drift-strength-aware dual-memory initialization procedure. First, given the current drift strength s_{drift}, the algorithm computes the mixing ratio $\rho(s_{drift})$ via a piecewise-linear mapping, thereby adaptively determining the relative contributions of the short-term buffer R_c and the long-term memory H_c to initialization under mild, moderate, and severe drift regimes. Next, on a per-class basis and under the capacity constraint sizesam, it samples without replacement $n_c^{(R)}$ and $n_c^{(H)}$ instances from R_c and H_c, respectively, to form an approximately class-balanced candidate set I_c. The sets $\{I_c\}$ are then merged and sorted by the arrival time t_i in ascending order to produce the global initialization sequence I, which remains reasonable in both class composition and temporal causality. The algorithm then performs a single pass over I. Building on the composite sample weight provided by PCN-Weight, it introduces a time-decay factor η_i, so that the sample difficulty, the class scarcity in the current stream, and the time distance to the drift moment are jointly integrated into the effective training weight $\widetilde{w}_i$. Finally, following the online bagging principle, for each sample it draws the replication count from Poisson($\widetilde{w}_i$) and completes the initialization of the candidate base classifier C_{new} through multiple incremental updates, enabling it to jointly accommodate recent distribution adaptation and long-term pattern inheritance before being deployed.

Algorithm 4 PDSR-based SoftReplace

Inputs:

E: current ensemble with D base classifiers $\{C_1, \ldots, C_D\}$

C_{new}: candidate classifier initialised by Algorithm 3

T_{trial}: length of the PDSR trial period

$\lambda_{\text{acc}}, \lambda_{\text{imb}}, \lambda_{\text{div}}$: utility weights

β: utility smoothing factor $(0 < \beta \leq 1)$

Output: Updated ensemble E(either with C_{new}kept or discarded).

Process (PDSR: performance–diversity driven soft replacement):

Softly insert the candidate classifier into the ensemble:

$E \leftarrow E \cup \{C_{\text{new}}\}$ and set the internal index of $C_{\text{new}}$to "new".

Initialise PDSR trial state: set trialCounter $\leftarrow 0$.

For each classifier $C_k \in E$do

    $U_k \leftarrow 0$.

    Reset sliding-window statistics $\hat{e}^{\,k}$, $\widehat{\text{GMean}}^{\,k}$, $\widehat{\text{Div}}^{\,k}$.

End for

While trialCounter $< T_{\text{trial}}$and new labelled samples are available do

    Receive a labelled instance (x, y).

    For each classifier $C_k \in E$do

        Obtain prediction $\hat{y}_k$and update windowed error rate $\hat{e}^{\,k}$.

        Update minority-aware metric $\widehat{\text{GMean}}^{\,k}$(or macro-F1) on the same window.

        Update diversity statistic $\widehat{\text{Div}}^{\,k}$(e.g. average disagreement with the ensemble prediction).

        Compute instantaneous utility

$U_k^{(\text{inst})} \leftarrow \lambda_{\text{acc}}(1 - \hat{e}^{\,k}) + \lambda_{\text{imb}}\widehat{\text{GMean}}^{\,k} + \lambda_{\text{div}}\widehat{\text{Div}}^{\,k}$.

        Update smoothed utility

$U_k \leftarrow (1 - \beta)\, U_k + \beta\, U_k^{(\text{inst})}$.

    End for

    trialCounter $\leftarrow$ trialCounter $+ 1$.

End while

Find the member with the lowest utility after the trial:

$k^* \leftarrow \arg\min_k U_k$.

If k^* is not the index of C_{new} then

    Replace $C_{k^*}$by $C_{\text{new}}$and keep ensemble size Dunchanged.

Else

    Discard $C_{\text{new}}$and restore E to its original Dmembers.

End if

Return E

Algorithm 4 presents the performance- and diversity-driven soft replacement strategy. After SDMI initialization, the newly constructed base classifier C_{new} is inserted into the current ensemble as a trial member, forming a temporary ensemble of size $D + 1$ together with the existing D base classifiers. Meanwhile, a sliding evaluation window with a unified length is maintained for each base classifier, on which three types of statistics are tracked, including the overall error rate $\widehat{e}^{(k)}$ to characterize global predictive accuracy, the minority-oriented performance metric $\widehat{GMean}^{(k)}$ to reflect adaptability to class imbalance, and the diversity measure $\widehat{Div}^{(k)}$ to quantify the disagreement between this member and the others in their predictions. Upon the arrival of each newly labeled instance, all base classifiers are incrementally updated as usual; simultaneously, the above statistics are refreshed on the same

Table 1 Characteristics of synthetic data streams

No	Datasets	Samples	Features	CIasses	Dirfts	Ib. Ratio	Drift type	Noise
1	VarImb_ISSI_2	300000	21	5	4	9/5/3/2/1 → 6/6/4/3/1 → 1/1.5/2/2.5/3	(RBF&inc.) sud. (Tree) sud. (RBF&inc.)	0
2	Var_Mix_SIG2	400000	20	5	3	28/10/6/5/1 → 1/4/4.7/4.7/19 → 1/4/4.7/4.7/19	(RBF)sud. (RBF&inc.)grad. (Tree)	0
3	Sta_Mix_SIG	400000	20	5	3	15/5/2/2/1	(RBF)sud. (RBF)inc. (Tree)grad	0
4	VarImb_SVg_2	300000	21	5	2	19:13:9:6:3 → 36:28:20:11:5 → 4:7:11:13:15	(RBF) sud. (Tree) grad. (Tree)	0
5	VarImb_SIS2	300000	21	5	3	6/4/3/2/1 → 12/6/4/3/1 → 12/6/4/3/1	(RBF)sud. (RBF&inc.)sud. (Tree)	0
6	VarImb_SIS	300000	21	5	3	5/4/3/2/1 → 10/5/5/4/1 → 10/5/5/4/1	(RBF) sud. (RBF&inc.) sud. (Tree)	0
7	Sudd_Var_1	200000	20	5	1	1/2/3/4/5 → 5/4/3/2/1	(RBF)sud	0
8	VarImb_ISSI	300000	21	5	4	5/4/3/2/1 → 5/1/2/2/5 → 10/5/5/4/1	(RBF&inc.) sud. (Tree) sud. (RBF&inc.)	0
9	Grad_Var_1	200000	20	5	1	1/2/3/4/5 → 5/4/3/2/1	(RBF)grad	0
10	VarImb_SVg	300000	21	5	2	5/4/3/2/1 → 5/4/3/2/1 → 1/2/3/4/5	(RBF) sud. (Tree) virt.&grad. (Tree)	0
11	Var_Mix_IGGI	300000	20	5	2	10:7:5:2:1 → 1:2:5:7:10 → 28:10:6:5:1	(RBF & inc.) grad (Tree) grad (RBF & inc.)	0
12	VarImb_GISGrN10_5	400000	21	5	4	10/5/5/4/1 → 1/2/3/4/5 → 5/1/2/2/5 → 10/5/5/4/1	(RBF)grad.(RBF&inc.)sud.(RBF) grad.&rec.(Tree)	10
13	VarImb_GISGrN10_6	400000	21	5	4	41/19/20/15/5 → 7/13/19/27/34 → 34/6/13/13/34 → 41/19/20/15/5	(RBF)grad.(RBF&inc.)sud.(RBF) grad.&rec.(Tree)sud	10
14	Var_Mix_SG	300000	20	5	2	1/2/3/4/5 → 5/4/3/2/1 → 1/2/3/4/5	(RBF)sud. (Tree)grad	0
15	Var_Mix_SIG	400000	20	5	3	1/2/2/5/15 → 15/5/2/2/1 → 1/2/2/5/15 → 20/5/2/2/1	(RBF)sud. (RBF)inc. (Tree)grad	0

Table 2 Characteristics of real data streams

No	Datasets	Samples	Features	CIasses	Dirfts	Ib. Ratio	Drift type	Noise
1	diabetes	253680	21	5	-	-	-	-
2	penbased	10092	16	10	-	-	-	-
3	banana	5300	2	2	-	-	-	-
4	letter	20000	16	26	-	-	-	-
5	ring	7400	20	2	-	-	-	-

sliding window, and the instantaneous utility is computed as $U_k^{(inst)} = \lambda_{acc}(1-\widehat{e}^{(k)}) + \lambda_{imb}\widehat{GMean}^{(k)} + \lambda_{div}\widehat{Div}^{(k)}$, after which an exponential moving average is applied to obtain the smoothed current utility U_k. This design allows each queried label to simultaneously drive model parameter updates and refine the utility evaluation along the three dimensions of accuracy, imbalance adaptation, and diversity, thereby maximizing label reuse under a fixed labeling budget. After the trial period ends, Algorithm 4 selects the index with the smallest utility among all $D+1$ members, $k^* = \text{argmin}_k U_k$. If $k^* \neq$ new, the candidate base classifier C_{new} is superior in overall utility to at least one existing member, and thus C_{new} replaces the lowest-utility old classifier C_{k^*}, restoring the ensemble size to D; if $k^* =$ new, the candidate model fails to provide sufficient gain under the current drift stage and is therefore discarded, leaving the original ensemble structure unchanged. Compared with hard replacement strategies that rely solely on error rate, this soft replacement mechanism explicitly considers global accuracy, minority-class performance, and predictive diversity when introducing a new model, effectively reducing the risk of catastrophic replacement caused by drift misdetection or short-term fluctuations, while preserving structural stability as the ensemble converges, thereby providing DMAE with a smoother and more reliable structural self-adaptation capability in long-horizon multiclass-imbalanced streams with mixed concept drift.

2.9 Comprehensive sample weighting with PCN-weight

This subsection presents PCN-Weight, a comprehensive sample-weighting formulation for DMAE. PCN-Weight jointly incorporates local prediction difficulty, intra-class structural information, class-imbalance status, and time decay into a unified weighting framework. In this way, it strengthens hard samples near decision boundaries, distinguishes typical patterns from tail samples within the same class, and limits the long-term dominance of majority classes during incremental training.

Given the sample (x, y) arriving at the current time, the ensemble classifier outputs the class posterior probability distribution for x. Let the class with the largest predicted probability be y_{c_1} and the second largest be y_{c_2}. Define the decision-boundary margin as $margin(x) = p(y_{c_1} \mid x) - p(y_{c_2} \mid x)$. To distinguish correctly predicted high-confidence samples, near-boundary samples, and clearly misclassified samples, we introduce indicator variables tf and s. According to the relationship among the true label y and y_{c_1}, y_{c_2}, they are defined as in Eq. (15).

$$\begin{aligned} tf = 1,\ s = 0, &\quad \text{when } y_{c_1} = y, \\ tf = -1,\ s = 1, &\quad \text{when } y_{c_2} = y, \\ tf = -1,\ s = 0, &\quad \text{when } y_{c_1} \neq y \text{ and } y_{c_2} \neq y. \end{aligned} \tag{15}$$

Based on this, the base difficulty term of the sample (x, y) is defined as in Eq. (16).

$$\mathrm{diff}_{base}(x, y) = (1 - tf \cdot margin(x)) \times \exp(1 - tf - s) \tag{16}$$

When a sample is correctly predicted with high confidence, $margin(x)$ is large and $tf = 1$, and thus $\mathrm{diff}_{base}(x, y)$ takes a small value; when a sample lies near the boundary or is misclassified, $margin(x)$ is small or even negative and $tf = -1$, and $\mathrm{diff}_{base}(x, y)$ will be significantly amplified, thereby emphasizing the hard samples that truly shape the decision boundary at the weighting level.

To characterize the class-imbalance status within the current label sliding window, let the window size be *sizelab*, the number of placeholders (i.e., positions where no label is requested) be *numNull*, the number of true labels of class y within the window be $\mathrm{labelnum}_y$, and the total number of classes be C. The imbalance indicator of class y is defined as in Eq. (17).

$$\mathrm{imb}_y = \frac{\mathrm{labelnum}_y}{(sizelab - numNull)/C} \tag{17}$$

When $\mathrm{imb}_y = 1$, the occurrence proportion of class y in the window is close to the average level; $\mathrm{imb}_y < 1$ indicates a relatively minority class, and $\mathrm{imb}_y > 1$ indicates a relatively majority class. This indicator provides a quantitative basis for subsequent class-quota control and minority amplification.

Relying on the short-term window and long-term memory structure constructed previously, a long-term prototype vector μ_c is maintained for each class c, which can be obtained by a weighted average over representative samples in the long-term memory set of that class. Let $h(x)$ denote the representation of sample x under the current ensemble (e.g., the last-layer latent vector). Then the Euclidean distance of (x, y) to the prototype of its class is defined as in Eq. (18).

$$d_{proto}(x, y) = \| h(x) - \mu_y \|_2 \tag{18}$$

To reflect the typical spatial scale of class y at the current stage, a smoothed average prototype distance $d_y(t)$ is maintained for each class, as in Eq. (19). Let the smoothing coefficient be $\lambda_d \in (0,1]$. When a new sample (x, y) arrives and $y = c$, update $d_y(t)$ by

$$d_y(t) = (1 - \lambda_d)\, d_y(t-1) + \lambda_d\, d_{proto}(x, y) \tag{19}$$

For other classes, $d_c(t)$ remains unchanged. Based on this, the prototype-contrast ratio is defined as in Eq. (20).

$$r(x, y) = \frac{d_{proto}(x, y)}{d_y(t)} \tag{20}$$

When $r(x, y) > 1$, the sample is significantly farther from the class prototype, typically corresponding to tail hard examples or structurally atypical samples; when $r(x, y) < 1$, the sample is closer to the prototype and represents a more typical intra-class pattern. Through $r(x, y)$, PCN-Weight distinguishes typical samples near the prototype from structural tail samples deviating from the prototype, and relatively amplifies the latter in weight allocation, so that under a limited labeling budget the method focuses more on instances that are more sensitive to intra-class boundary geometry.

Relying only on the instantaneous proportions within the sliding window may still cause short-term high-frequency majority classes to continuously occupy excessive training capacity over a long horizon. To explicitly introduce a class-level quota constraint into the sample-weight design, PCN-Weight defines a quota adjustment factor $q(y)$ for each class based on imb_y, as in Eq. (21).

$$q(y) = 1 + \lambda_q \left(\frac{1}{\mathrm{imb}_y} - 1 \right) \tag{21}$$

where $\lambda_q \in (0,1)$ controls the adjustment magnitude, and in implementation $q(y)$ is clipped to the interval $[q_{min}, q_{max}]$, where $0 < q_{min} < 1 < q_{max}$. When $\mathrm{imb}_y > 1$, the quota factor $q(y) < 1$, and the total effective weight of this class over a period is mildly compressed; when $\mathrm{imb}_y < 1$, $q(y) > 1$, reserving more effective training quota for minority classes,

thereby suppressing the long-term dominance of majority classes in parameter updates over the stream.

Considering that in streaming scenarios sample usefulness gradually decays, each sample involved in subsequent training and updates records its arrival time $t_{arriving}$, so that time decay can be introduced in PCN-Weight and in the dual-memory initialization process. Let the current time step be $t_{current}$. The time-decay factor is defined as in Eq. (22).

$$factor(x) = \exp\left(-\gamma \cdot \frac{t_{current} - t_{arriving}}{sizelab}\right) \tag{22}$$

where $\gamma > 0$ controls the decay rate. This factor makes the weights of samples close to the current time approach 1, while the weights of older historical samples exponentially decay with time, which better matches the temporal evolution of concepts and prevents outdated samples from exerting overly strong influence after drift.

Combining the above four components, PCN-Weight integrates base difficulty, intra-class prototype contrast, class-level quota, and time decay into an overall weighting function. Given a sample (x, y), the final training weight is defined as in Eq. (23).

$$weight_{PCN}(x,y) = \log\left(1 + \mathrm{diff}_{base}(x,y) + \frac{1}{\mathrm{imb}_y}\right) \cdot \left[1 + \lambda_p(r(x,y) - 1)\right] \cdot q(y) \cdot factor(x) \tag{23}$$

where $\lambda_p \in [0,1]$ controls the strength of the prototype-contrast term. In the final training weight, the first part characterizes classification difficulty in the margin space and further amplifies minority samples via $1/\mathrm{imb}_y$; the second part uses $r(x, y)$ to adaptively identify and emphasize intra-class tail samples far from the prototype, so that samples that truly affect intra-class boundary geometry receive higher weights; the third part imposes a soft quota at the class level according to imbalance status to prevent majority classes from dominating training over long horizons; and the fourth part uses time decay to keep the model focused on the recent distribution. With this comprehensive weighting design, DMAE focuses on hard and critical samples locally, and balances the effective contribution of each class across time globally, providing more robust and discriminative gradient signals for the subsequent active labeling strategy and drift adaptation mechanisms.

2.10 Time and space complexity analysis

We analyze the time and space complexity of the proposed DMAE algorithm as follows. Let the stream length be n, the number of base classifiers in the ensemble be D, and the number of classes be C. Denote the per-sample prediction cost and the per-sample incremental update cost of a single base classifier as T_{pred} and T_{upd}, respectively. Let the label sliding window length be W_{lab}; for each class c, let the capacity of the short-term sample window R_c be W_{short}; let the maximum number of prototypes per class stored in the long-term prototype memory H_c be P_{proto}; and let the length of the PDSR trial period be T_{trial}. For the space analysis, let the model size upper bound of a single base learner be S_{base}.

(1) Time Complexity: For each arriving unlabeled instance, DMAE first performs ensemble prediction over D base classifiers and computes the uncertainty information used by BAM^2-Lite. The prediction-stage cost is $O(D \cdot T_{\mathrm{pred}})$. The class counts in the label sliding window are maintained incrementally, so estimating the current class distribution and imbalance state does not require scanning the whole stream. The BAM^2-Lite decision only involves the current top-1 and top-2 predicted classes, the corresponding class-pair state, the current drift-strength scalar, and the current budget ratio. Therefore, its per-sample decision cost is constant with respect to the stream length n. Under a fixed labelling budget b, only a fraction of arriving instances can trigger oracle labelling and supervised updating. For a queried instance, DMAE computes the PCN-Weight, updates the corresponding short-term sample window and long-term prototype memory, updates the BAM^2-Lite state if the sample falls into the corridor, and incrementally updates the ensemble. The dominant supervised update cost is $O(D \cdot T_{\mathrm{upd}})$. Therefore, when no drift response is triggered, the expected per-sample time cost can be written as $O(D \cdot T_{\mathrm{pred}} + bD \cdot T_{\mathrm{upd}})$, where the factor b reflects that supervised updating is performed only on budgeted queried samples. Since b is fixed in the experiments, the regular online processing cost grows linearly with the stream length. When ADWIN reports significant concept drift, the SDMI-PDSR subroutine is triggered. In SDMI, for each class, at most W_{short} samples from the short-term window and P_{proto} prototypes from the long-term memory are used to construct the initialization sequence. The cost of initializing the candidate base classifier is therefore $O\big(C(W_{\mathrm{short}} + P_{\mathrm{proto}}) \cdot T_{\mathrm{upd}}\big)$. During the PDSR trial period, the candidate classifier is evaluated together with the existing ensemble on labelled samples. The trial-stage overhead is bounded by $O\big(T_{\mathrm{trial}}(D \cdot T_{\mathrm{pred}} + T_{\mathrm{upd}})\big)$. Let N_{drift} denote the number of detected drift events over the stream. The total time complexity can be expressed as $O\big(n(D \cdot T_{\mathrm{pred}} + bD \cdot T_{\mathrm{upd}})\big) + O\big(N_{\mathrm{drift}} \cdot C(W_{\mathrm{short}} + P_{\mathrm{proto}}) \cdot T_{\mathrm{upd}}\big)$

$+O\big(N_{\text{drift}} \cdot T_{\text{trial}}(D \cdot T_{\text{pred}} + T_{\text{upd}})\big)$. This expression shows that the main online cost is linear in n, while the additional SDMI-PDSR cost is incurred only when drift is detected. In long-horizon streams, DMAE does not revisit historical samples and does not store the full stream. Therefore, the regular processing cost remains linear. Under frequent drift, the extra cost is controlled by the fixed memory capacities W_{short} and P_{proto}, the fixed ensemble size D, and the bounded trial length T_{trial}. Consequently, even in drift-intensive streams, the additional cost is bounded by the number of detected drift events and does not require full-history retraining.

(2) Space Complexity: The major memory consumption of DMAE comes from the ensemble model, the label sliding window, the class-wise short-term sample windows, the long-term prototype memories, and the BAM^2-Lite state variables. For a given base-learner family, storing D base classifiers requires $O(D \cdot S_{\text{base}})$ space. The label sliding window of length W_{lab} requires $O(W_{\text{lab}})$ space. The short-term windows and long-term prototype memories are maintained for each class, resulting in the memory cost $O\big(C(W_{\text{short}} + P_{\text{proto}})\big)$. The BAM^2-Lite state includes the class-level potential vector and class-pair interaction statistics. These structures have dimensions no larger than C or $C \times C$, leading to $O(C^2)$ space. Other auxiliary variables, such as the drift-strength index, budget counters, ADWIN statistics, and PDSR utility statistics, require only bounded storage with respect to the stream length. Therefore, the overall space complexity of DMAE is $O\big(D \cdot S_{\text{base}} + W_{\text{lab}} + C(W_{\text{short}} + P_{\text{proto}}) + C^2\big)$. Since D, C, W_{lab}, W_{short}, P_{proto}, and T_{trial} are fixed hyperparameters independent of the stream length n, the space usage of DMAE does not grow with n. This is important for long-horizon streaming scenarios: DMAE maintains only bounded sliding windows, bounded prototype memories, fixed-size ensemble members, and fixed-size class-pair statistics, rather than storing all historical data. Thus, the method is practical for continuous data streams with limited memory resources.

3 Experimental evaluations

This section presents the experimental study in detail. Section 4.1 introduces the datasets and evaluation protocol. Section 4.2 describes the implementation details and hyperparameter settings of DMAE and the competing methods. Section 4.3 discusses the reproducibility and fairness of the experimental comparisons to ensure that all algorithms are evaluated under a consistent environment. Section 4.4 compares DMAE with other methods using multiple evaluation metrics. Section 4.5 reports the runtime behavior of the algorithms using the average value over all evaluation metrics. Section 4.6 presents an overall multi-metric comparison and an average-rank performance analysis to further assess the advantages of DMAE. Section 4.7 reports an ablation study to examine the contribution of each component in DMAE. Section 4.8 conducts a parameter sensitivity analysis to evaluate DMAE under different parameter settings and to determine appropriate default values. Section 4.9 performs statistical analysis across all algorithms. All learning algorithms are implemented in the Massive Online Analysis (MOA) data stream software suite. The experiments are conducted on a machine running Windows 11, equipped with an R7-7840H CPU at 3.80 GHz and 32 GB of memory. Unless otherwise specified, all compared methods are evaluated under the same prequential protocol, the same global labelling budget, and the same stream order.

3.1 Datasets and evaluation protocol

To comprehensively evaluate DMAE under both controlled and realistic non-stationary environments, we use 15 synthetic multiclass imbalanced data streams and 5 real-world imbalanced data streams. The synthetic streams are mainly used to compare competing methods under controlled conditions, where the multiclass imbalance ratio, the number of classes, the drift type, and the drift speed can be explicitly configured to generate diverse stream scenarios. Following this principle, we use the RandomRBF and RandomTree generators provided by the Massive Online Analysis (MOA) suite to construct the synthetic streams used in the experiments.

Tables 1 and 2 summarize the characteristics of the synthetic streams. For readability, we report the most important properties of each stream, including the stream size, feature dimensionality, number of classes, number of drifts, imbalance setting, drift pattern, and noise level. Among these synthetic streams, Sta_Mix_SIG is the only stream with a fixed imbalance ratio (FixedImb), whereas all the remaining synthetic streams follow variable imbalance ratios (VarImb). The drift settings cover sudden, gradual, incremental, and mixed drift, and also include streams with both real drift and virtual drift. In all synthetic streams, each concept persists for 100,000 instances. Moreover, Streams 12 and 13 additionally contain label noise. Overall, among the 15 synthetic streams, seven have 20 attributes and eight have 21 attributes. The exact generator configurations and ratio trajectories are provided in the project scripts. Table 2 lists the real-world data streams. These streams are obtained from the KEEL machine-learning repository and cover different class cardinalities and imbalance conditions. Unlike the synthetic streams, the exact drift type and the number of drift

Table 3 Accuracy results of comparative experiments (%)

20 Datasets/11 Algorithms		ROSE	CALMID	AdaOB	OALM-MI	KUE	MicFoal	CSMOTE	SRP	MOOB	CSARF	**DMAE**
Synthetic Datasets	Mix_SIG_Var2	94.33	95.71	94.39	95.46	91.27	83.18	63.18	92.99	89.99	74	**97.12**
	Grad_Var_1	90.48	94.78	90.69	95.72	88.4	78.08	64.02	92.34	84.22	66.31	**97.53**
	VarImb_ISSI_2	85.80	88.94	87.33	87.51	79.54	61.34	43.70	87.14	80.57	50.73	**92.78**
	VarImb_SVg_2	91.89	92.00	92.14	92.57	92.03	75.01	56.37	93.93	88	53.41	**97.87**
	Mix_SG_Var	91.06	94.75	93.85	95.03	92.18	81.66	63.71	90.51	90.01	64.08	**97.84**
	Mix_SIG_Sta	93.83	94.47	92.68	93.98	91.11	81.65	67.16	91.26	87.96	83.46	**96.44**
	Mix_SIG_Var	94.33	95.59	93.88	94.72	89.35	78.68	69.42	88.18	87.97	77.26	**96.79**
	Sudd_Var_1	92.35	96.28	90.63	95.04	89.34	74.98	69.42	93.32	84.28	66.75	**97.80**
	Var_Mix_IGGI	92.29	92.36	92.14	92.03	87.40	77.04	56.99	92.76	84.51	62.78	**94.78**
	VarImb_SIS2	88.11	91.59	84.3	87.62	84.9	65.43	52.59	85.88	77.75	58.01	**92.50**
	VarImb_SIS	91.11	93.14	91.4	93	86.19	73.7	51.65	89.62	83.99	62.86	**95.96**
	VarImb_ISSI	87.39	88.72	90.76	88.32	82.56	69.06	44.98	89	76.43	66.73	**93.31**
	VarImb_SVg	91.67	96.43	93.84	94.29	91.83	77.9	51.41	88.26	82.98	53.48	**98.27**
	VarImb_GISGrN10_5	89.37	91.59	89.33	91.42	87.43	77.68	46.29	87.55	84.69	60.49	**94.46**
	VarImb_GISGrN10_6	89.25	91.46	89.2	91.58	86.67	74.81	46.15	88.22	85.12	60.49	**94.52**
Real Datasets	penbased	89.53	89.53	90.93	88.07	57.67	85.6	78.15	87.68	88.82	91.31	**92.33**
	banana	85.67	68.67	85.9	73.3	75.5	62.4	46.77	86.87	66.93	50.67	**88.72**
	letter	65.19	61.71	64.01	49.18	55.53	63.92	25.25	60.63	63.82	33.68	**72.94**
	ring	97.27	93.73	97.55	98.07	90.1	98.35	80.08	95.82	96	80.73	**98.54**
	diabetes	43.93	43.96	42.71	44.06	37.78	40.73	32.36	**44.25**	42.15	29.14	44.12

Table 4 Kappa results of comparative experiments (%)

20 Datasets/11 Algorithms		ROSE	CALMID	AdaOB	OALM-MI	KUE	MicFoal	CSMOTE	SRP	MOOB	CSARF	**DMAE**
Synthetic Datasets	Var_Mix_SIG	90.73	92.97	91.02	92.49	84.59	72.37	43.62	88.18	83.49	55.07	**95.12**
	Grad_Var_1	87.32	87.71	87.67	94.87	85.28	71.48	52.7	89.79	79.47	54.41	**96.80**
	VarImb_ISSI_2	80.51	83.60	82.97	82.40	67.20	41.33	22.43	81.98	71.99	31.15	**90.26**
	VarImb_SVg_2	88.86	89.05	89.34	91.87	89.04	65.46	42.31	92.07	84.34	31.76	**97.10**
	Mix_SG_Var	88.35	93.01	91.91	93.48	89.63	75.85	51.41	87.36	86.85	52.23	**97.16**
	Mix_SIG_Sta	93.83	90.29	87.78	89.51	83.83	67.02	40.45	83.99	79.21	74.95	**93.70**
	Mix_SIG_Var	89.97	92.17	89.55	90.67	80.76	63.57	43.62	76.16	79.82	58.35	**94.14**
	Sudd_Var_1	89.82	95.1	87.54	93.44	86.06	67.38	43.62	91.14	79.57	51.56	**97.10**
	Var_Mix_IGGI	89.78	89.24	89.19	88.57	81.45	67.22	36.72	90.06	78.19	45.55	**92.15**
	VarImb_SIS2	83.21	82.42	78.03	82.52	77.86	48.72	32.99	79.98	69.91	37.46	**90.24**
	VarImb_SIS	87.88	90.59	88.32	90.43	80.54	63.35	32.56	85.57	77.65	46.88	**94.44**
	VarImb_ISSI	82.98	84.38	87.65	84.46	76.48	57.95	25.57	84.81	68.88	53.61	**90.96**
	VarImb_SVg	89.02	95.26	91.82	92.44	89.03	70.42	31.29	83.77	77.06	38.12	**97.70**
	VarImb_GISGrN10_5	85.67	88.18	85.7	88.37	82.72	69.86	28.59	82.72	79.41	43.29	**92.50**
	VarImb_GISGrN10_6	85.48	88.4	85.44	88.58	81.43	66.39	46.15	83.63	79.9	42.95	**92.55**
Real Datasets	penbased	88.34	88.04	89.9	86.72	53.04	83.95	75.59	87.95	87.55	58.36	**91.46**
	banana	71.28	53.48	71.68	45.69	49.01	22.12	0	73.53	32.85	1.28	**76.35**
	letter	63.75	60.18	62.52	47.05	53.71	62.42	22.21	58.99	62.23	30.99	**71.78**
	ring	94.55	94.63	95.07	96.15	80.36	96.7	92.38	91.69	92.38	61.41	**96.92**
	diabetes	**25.05**	21.62	21.82	20.41	22.37	16.96	9.52	22.86	22.76	13.31	23.25

Table 5 G-Mean results of comparative experiments (%)

20 Datasets/11 Algorithms		ROSE	CALMID	AdaOB	OALM-MI	KUE	MicFoal	CSMOTE	SRP	MOOB	CSARF	**DMAE**
Synthetic Datasets	Var_Mix_SIG	88.02	90.07	89.09	88.81	77.87	70.57	34.72	83.94	83.87	59.3	**92.27**
	Grad_Var_1	89.85	89.71	89.52	94.87	86.11	76.21	57.74	89.9	83.39	55.82	**96.98**
	VarImb_ISSI_2	85.15	84.42	86.65	86.02	63.85	43.46	17.82	54.15	76.26	40.10	**90.16**
	VarImb_SVg_2	90.26	92.20	90.27	93.73	87.46	67.62	47.81	92.83	87.26	37.24	**96.41**
	Mix_SG_Var	87.75	91.42	91.94	93.01	90.02	74.39	49.93	87.8	87.38	40.48	**97.05**
	Mix_SIG_Sta	87.39	87.44	87.82	88.51	77.42	63.76	32.25	79.48	78.1	68.55	**90.27**
	Mix_SIG_Var	88.57	89.14	89.65	84.49	69.09	62.34	34.72	60.67	79.59	61.06	**91.08**
	Sudd_Var_1	92.12	95.43	89.16	94.5	86.06	72.16	34.72	91.94	83.53	48.74	**97.17**
	Var_Mix_IGGI	83.97	82.32	85.64	84.19	70.87	58.40	24.79	82.27	75.84	51.30	**86.82**
	VarImb_SIS2	84.61	83.13	82.63	85.23	77.15	53.03	31.01	78.57	75.12	43.62	**90.44**
	VarImb_SIS	88.39	90.55	89.09	91.36	78.87	68.3	34.18	84.86	79.78	53.49	**93.53**
	VarImb_ISSI	80.37	85.3	89.1	85.86	73.87	58.09	26.88	82.99	70.52	54.87	**90.26**
	VarImb_SVg	90.24	95.74	91.04	93.68	88.97	72.08	28.00	35.52	80.06	40.09	**97.70**
	VarImb_GISGrN10_5	85.57	87.32	86.32	87.5	77.31	70.13	27.64	77.57	78.98	49.89	**90.80**
	VarImb_GISGrN10_6	86.15	88.01	86.59	88.48	78.54	66.38	29.58	81.65	80.1	50.01	**91.45**
Real Datasets	penbased	88.4	88.96	90.67	86.31	45.37	83.38	75.89	88.69	88.02	41.77	**92.20**
	banana	85.55	54.8	85.86	70.73	64.1	56.27	0	85.65	65.47	43.79	**88.90**
	letter	61.73	57.68	60.91	38.05	48.27	60.66	1.48	52.53	60.38	13.05	**70.94**
	ring	97.27	93.73	97.54	97.16	88.97	96.7	79.74	95.93	95.64	80.71	**97.84**
	diabetes	**45.3**	37.7	39.08	15.33	33.37	25.42	18.75	38.43	43.45	27.35	44.08

Table 6 Recall results of comparative experiments (%)

20 Datasets/11 Algorithms		ROSE	CALMID	AdaOB	OALM-MI	KUE	MicFoal	CSMOTE	SRP	MOOB	CSARF	**DMAE**
Synthetic Datasets	Var_Mix_SIG	86.08	89.66	86.9	87.23	83.8	78.31	56.55	79.38	85.93	67.55	**91.22**
	Grad_Var_1	86.53	86.56	86.31	93.48	86.48	71.67	63.18	93.03	82.93	66.85	**96.30**
	VarImb_ISSI_2	85.00	85.59	85.19	85.93	55.75	63.19	66.21	89.82	78.18	58.31	**94.26**
	VarImb_SVg_2	93.72	88.26	90.99	92.30	92.06	76.20	60.43	92.48	85.42	56.40	**98.17**
	Mix_SG_Var	86.24	89.21	89.74	90.89	89.9	73.29	62.25	83.13	84.99	60.63	**97.40**
	Mix_SIG_Sta	95.33	95.34	94.38	94.08	91.75	80.49	70.16	90.91	87.26	80.34	**97.02**
	Mix_SIG_Var	88.42	91.55	91.47	84.71	73.81	65.86	56.52	64.38	78.15	73.21	**93.60**
	Sudd_Var_1	90.93	95.13	86.45	91.75	86.51	67.8	56.51	94.39	83.14	66.23	**96.87**
	Var_Mix_IGGI	88.22	91.87	90.78	88.84	79.34	73.01	58.63	88.45	77.96	62.32	**95.09**
	VarImb_SIS2	88.97	90.88	82.03	87.65	87.31	65.95	62.59	87.12	78.12	55.05	**93.82**
	VarImb_SIS	90.47	92.35	90.85	91.77	85.61	72.1	63.49	90.3	81.34	65.11	**96.14**
	VarImb_ISSI	77.86	87.84	88.59	84.64	76.31	65.96	57.67	83.28	66.29	65.28	**92.14**
	VarImb_SVg	83.64	94.8	90.39	89.1	89.54	71.3	59.33	88.41	80.12	59.9	**98.17**
	VarImb_GISGrN10_5	86.66	87.68	86.89	85.15	84.17	75.19	61.01	83.23	82.58	61.99	**90.59**
	VarImb_GISGrN10_6	86.7	87.08	86.39	85.27	83.82	76.18	62.64	83.49	82.46	61.38	**90.93**
Real Datasets	penbased	83.17	83.04	82.05	82.45	57.41	81.42	83.36	82.56	78.24	74.29	**86.61**
	banana	85.66	67.17	85.85	72.69	74.37	60.81	50	85.3	68.59	50.09	**88.92**
	letter	76.15	71.31	74.55	51.15	73.01	72.79	73.99	70.85	74.78	59.36	**81.56**
	ring	97.45	97.21	97.59	97.15	90.2	96.42	80.75	95.67	95.2	81.16	**98.04**
	diabetes	46.44	46.58	42.53	32.27	45.07	41.29	45.89	43.51	43.31	46.07	**47.24**

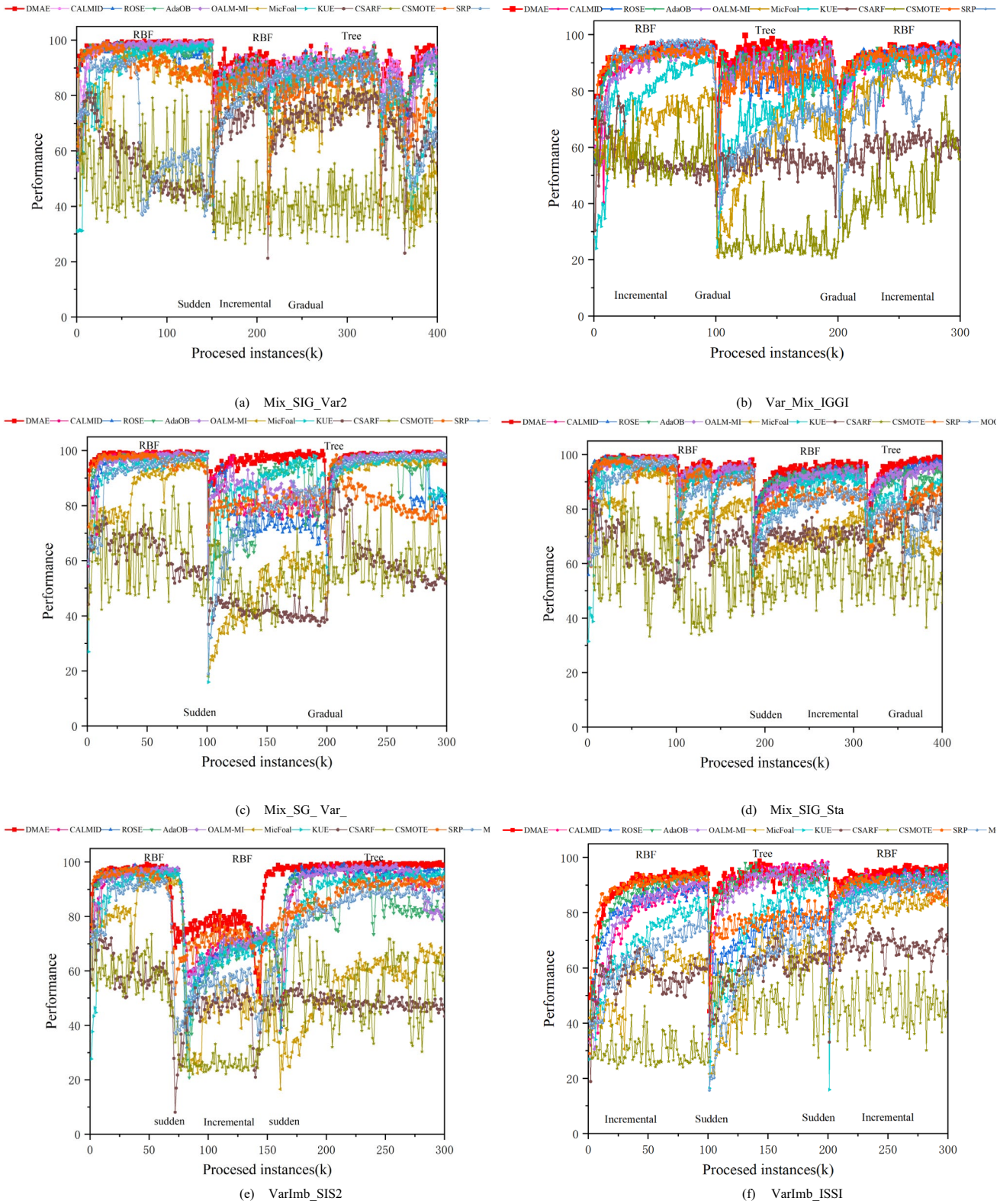


Fig. 2 Comprehensive evaluation metric

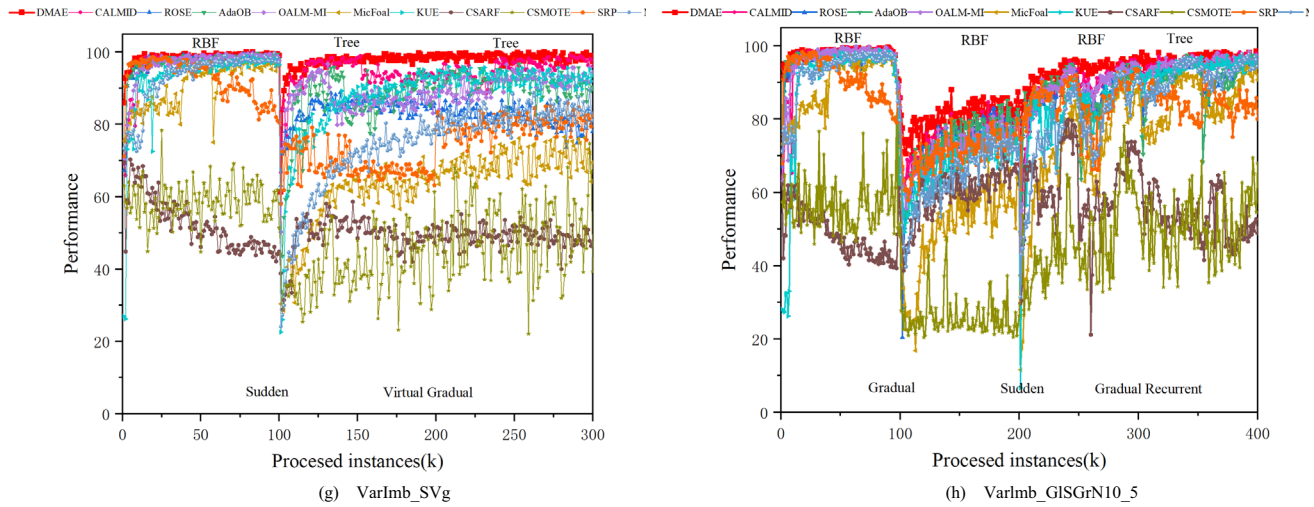


Fig. 2 (continued)

points in real-world streams are generally unknown a priori. Therefore, they are used to complement the synthetic evaluation by assessing the practical effectiveness of DMAE when the stream structure is less regular and the drift mechanism is not explicitly controlled. For the real-world datasets, each dataset is converted into a data stream by reading instances sequentially from the dataset file. Unless otherwise specified, the original instance order is preserved to avoid introducing artificial distributional changes by additional reshuffling. Categorical attributes, if any, are encoded according to the preprocessing interface of MOA or the corresponding data loader, and numerical attributes are used in their original feature representation. The class labels are only revealed to the algorithm when an instance is selected for annotation under the active-learning protocol. Therefore, even though the true labels are available offline for evaluation, they are hidden from the learner unless the global labelling budget permits a query. Unless otherwise stated, all methods are evaluated under the same interleaved test-then-train, namely prequential, protocol. At each time step, the current model first predicts the label of the arriving instance. The prediction is recorded for evaluation, and the model is then updated only if the instance is selected for annotation under the prescribed labelling budget. The default labelling budget is set to 20% for all datasets unless otherwise specified. This 20% budget is a global budget shared by random querying, BAM^2-Lite active querying, selective sampling, and drift-stage exploration, rather than a separate budget assigned to each query branch. This evaluation protocol is consistent with the online active-learning scenario considered in this paper, where limited queried labels must simultaneously support distribution estimation, weighted model updating, memory maintenance, and drift adaptation.

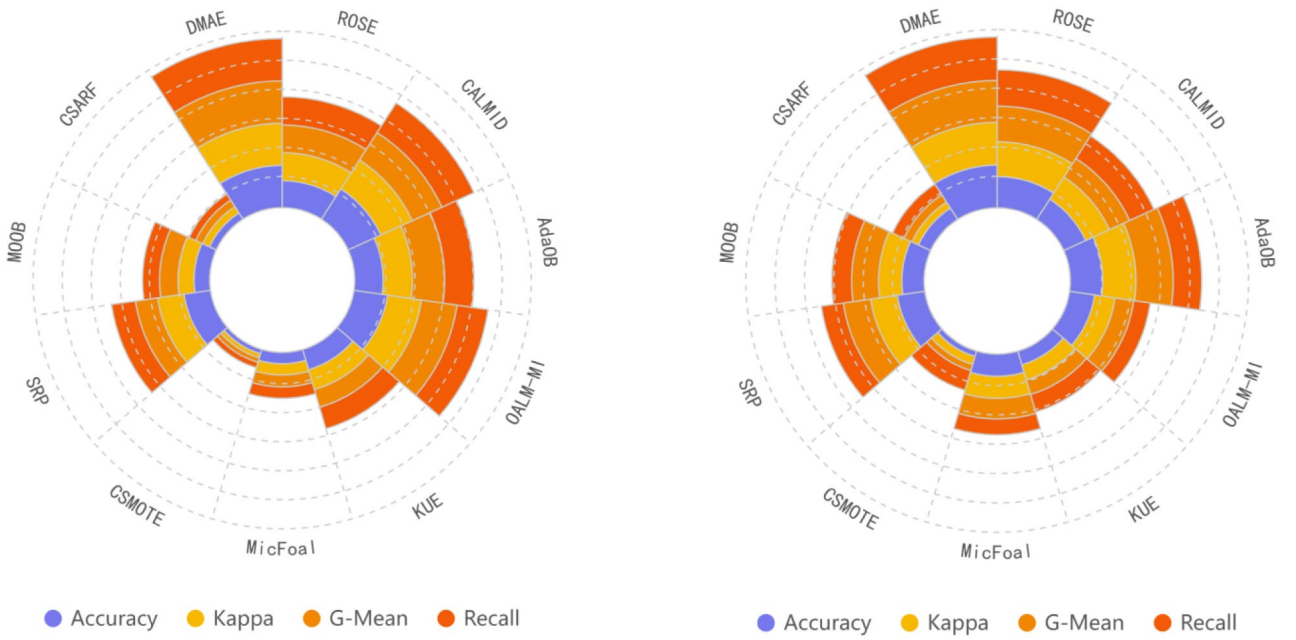
Evaluation measures for multiclass imbalanced data streams can be broadly divided into class-wise metrics and integral metrics (Brzezinski et al. 2020). Class-wise metrics, such as Recall, Accuracy, and the corresponding F-score, characterize a model's recognition ability and error structure from the perspective of individual classes. By contrast, integral metrics aggregate multiclass performance into a single indicator, such as G-Mean, and place greater emphasis on maintaining balanced recognition quality across classes.

It should be noted that Accuracy and F-score are often sensitive to class priors. When class proportions fluctuate over time, these measures may vary substantially even if the model's intrinsic discriminative ability does not change materially, thereby producing apparent performance oscillations caused by distribution shifts rather than true learning dynamics. In comparison, Recall directly measures the coverage of correctly recognized instances for the target class and is therefore more interpretable in multiclass imbalanced settings with asymmetric costs. Accordingly, a reliable online learner should maintain both high and stable recall performance over time.

Based on these considerations, we use Accuracy, Kappa, G-Mean, Recall, and their overall average performance as the main evaluation metrics. Accuracy, Kappa, G-Mean, and Recall are updated instance by instance during online learning, whereas the overall average performance is computed after training by aggregating the prediction records over the full online process.

Table 7 Average ranks of the compared algorithms across different metrics and stream types

Metrics	Stream Types	Algorithms										
		ROSE	CALMID	AdaOB	OALM-MI	KUE	MicFoal	CSMOTE	SRP	MOOB	CSARF	DMAE
Accuracy	Synthetic	5.00	2.67	4.60	3.20	6.53	9.07	10.87	5.07	7.93	10.07	1.00
	Real	4.10	5.90	3.80	5.60	8.40	6.40	10.60	5.00	6.40	8.60	1.20
	Total Rank	4.78	3.48	4.40	3.80	7.00	8.40	10.80	5.05	7.55	9.70	1.05
Kappa	Synthetic	4.73	3.07	4.40	3.13	6.63	9.07	10.87	5.10	7.87	10.07	1.07
	Real	3.20	5.40	3.60	6.80	8.00	6.40	9.90	5.20	6.10	10.20	1.20
	Total Rank	4.35	3.65	4.20	4.05	6.97	8.40	10.63	5.13	7.43	10.10	1.10
G-Mean	Synthetic	4.80	3.53	3.67	2.67	7.20	8.93	10.80	6.20	7.20	10.00	1.00
	Real	3.00	6.40	2.60	7.20	8.20	6.80	10.40	5.00	5.20	9.80	1.40
	Total Rank	4.35	4.25	3.40	3.80	7.45	8.40	10.70	5.90	6.70	9.95	1.10
Recall	Synthetic	4.67	2.87	4.40	3.87	6.20	9.00	10.53	5.73	7.60	10.13	1.00
	Real	2.80	5.20	4.80	7.80	7.40	8.00	6.80	6.40	7.00	8.80	1.00
	Total Rank	4.20	3.45	4.50	4.85	6.50	8.75	9.60	5.90	7.45	9.80	1.00
Meta rank		4.42	3.71	4.13	4.13	6.98	8.49	10.43	5.49	7.28	9.89	1.06



(a) Synthetic data streams

(b) Real data streams

Fig. 3 Wind-rose plots of algorithms on accuracy, kappa, g-mean and recall

3.2 Implementation details and hyperparameter settings

All experiments are implemented using the Massive Online Analysis (MOA) framework, which is widely used for evaluating online learning algorithms on evolving data streams. The proposed DMAE framework contains several key components, including the BAM^2-Lite hybrid querying mechanism, the drift-strength-aware dual-memory initialization (SDMI), the PDSR soft replacement strategy, and the PCN-Weight sample-weighting formulation. The main hyperparameters of DMAE include the baseline uncertainty threshold θ, the mixing weight β in BAM^2-Lite, the marginal corridor width δ, the upper bound of the short-term memory

ratio under strong drift ρ_max, and the global scaling factor of long-term prototypes λ_long. The candidate ranges of these parameters were determined through preliminary pilot experiments and common practices in online active learning. A small grid search on representative streams was conducted, and the configuration yielding the best prequential G-Mean performance was selected. The default parameter values used in all experiments are $\theta = 0.50$, $\beta = 0.50$, $\delta = 0.05$, $\rho_{max} = 0.80$, and $\lambda_{long} = 0.80$. A detailed sensitivity analysis of these parameters is reported in Section 4.7.

To ensure reproducibility, the same parameter configuration is used for DMAE on all datasets unless otherwise specified in the parameter sensitivity analysis. The ensemble size is fixed for all ensemble-based methods, and the Hoeffding Tree is used as the base classifier to ensure that the comparison focuses on the learning strategy rather than on different base learners. The data block size is set to 500, and the number of initial instances for active-learning methods is also set to 500. The default global labelling budget is fixed at 20%, and all query branches in DMAE are constrained

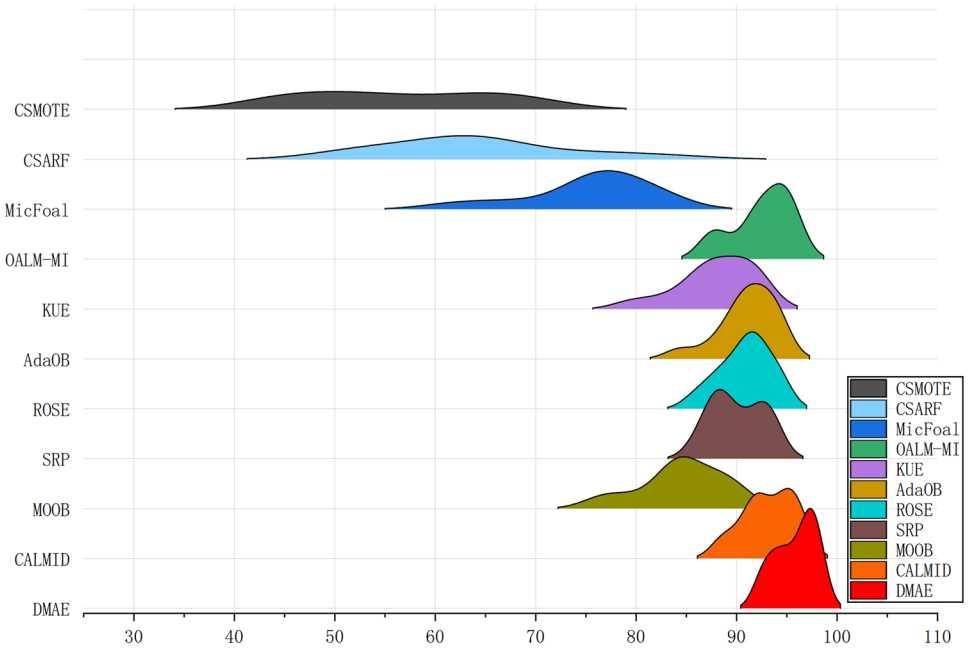


Fig. 4 Ridge plot of each model on all synthetic datasets (Accuracy)

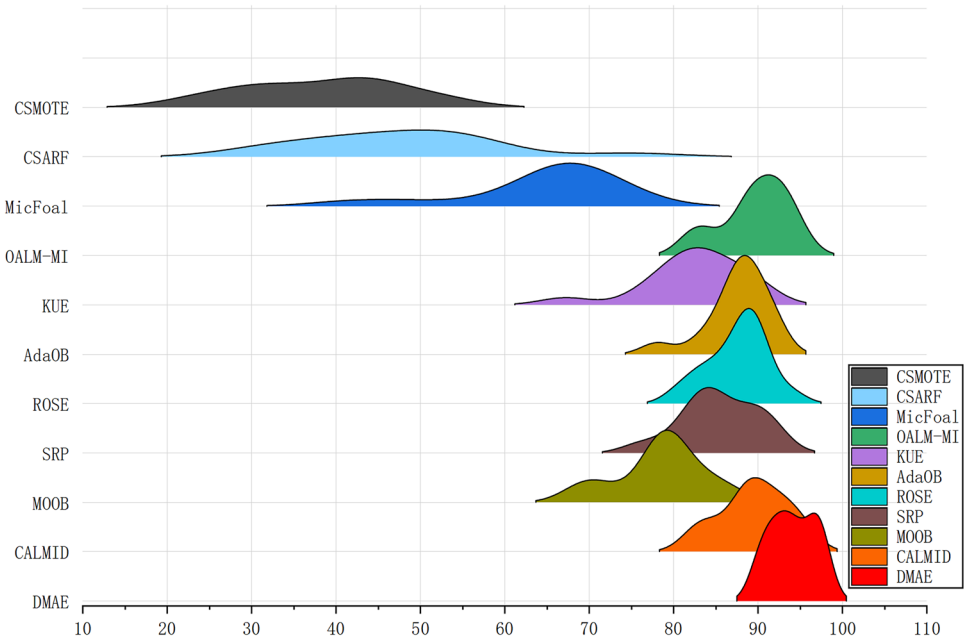


Fig. 5 Ridge plot of each model on all synthetic datasets (Kappa)

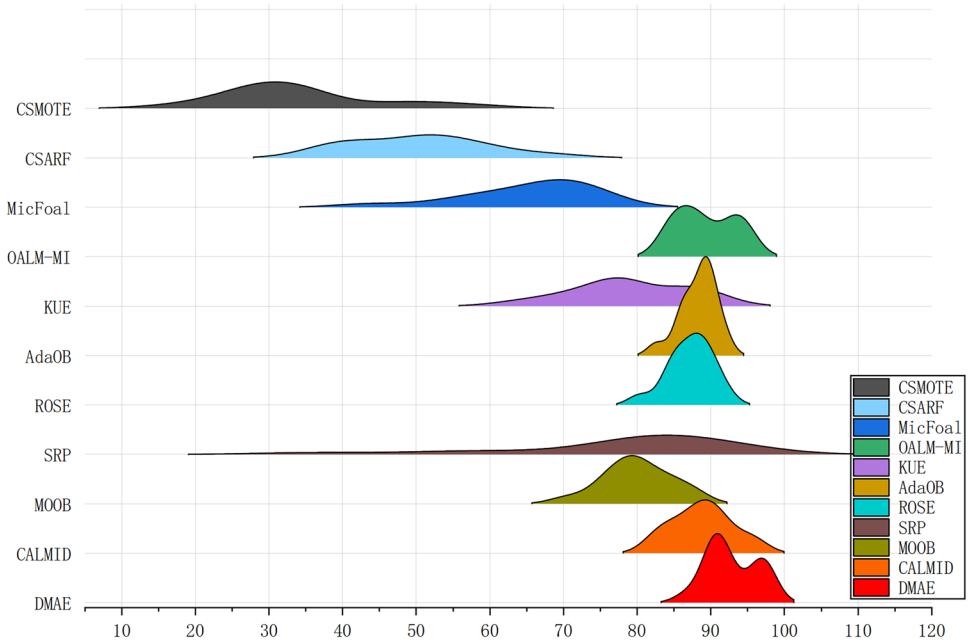


Fig. 6 Ridge plot of each model on all synthetic datasets (G-Mean)

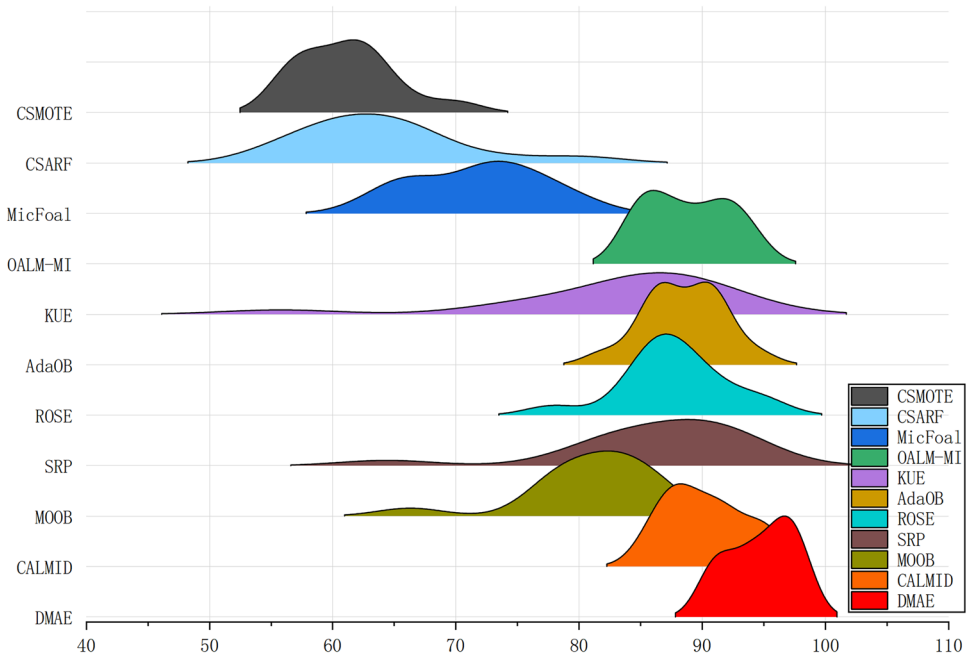


Fig. 7 Ridge plot of each model on all Synthetic datasets (Recall)

by this single budget. In other words, random querying, BAM^2-Lite active querying, selective sampling, and drift-stage exploration do not use independent label budgets. Each experiment is repeated with multiple independent random seeds, and the reported results correspond to the average performance over repeated runs. The random seeds are used for random querying, selective sampling, stream generators, and other stochastic components whenever applicable. To support reproducibility, all compared methods are executed under the same computing environment, identical evaluation protocol, identical stream configuration, and identical labelling budget. For baseline methods, whenever official MOA implementations are available, we directly use them; otherwise, we implement the methods according to their original descriptions and recommended parameter settings. Minor parameter adjustments are made only when necessary to make all methods compatible with the same online active-learning protocol.

For the baseline methods, whenever possible we use the official implementations provided in the MOA framework.

For algorithms not directly available in MOA, we implement them following the descriptions in the original publications. The baseline algorithms are configured using the recommended parameter settings reported in their respective papers, and minor adjustments are made when necessary so that all methods operate under the same online active-learning protocol and labeling budget. All experiments are conducted on the same computing environment under identical evaluation settings. Each experiment is repeated with multiple random seeds, and the reported results correspond to the average performance across runs. To facilitate reproducibility, the source code, parameter configuration files, and scripts for reproducing all reported results will be made publicly available upon acceptance of this paper.

3.3 Reproducibility and fairness of comparison

Some of the baseline algorithms considered in this study have previously reported experimental results on related data streams in the literature. However, the experimental configurations used in those studies are not always identical to the setting adopted in this work. In particular, differences may arise in the evaluation protocol, labeling budget, stream generation settings, and preprocessing procedures. To ensure a fair and reproducible comparison, we do not directly reuse the performance numbers reported in previous publications. Instead, all baseline algorithms are re-evaluated under a unified experimental environment that is consistent with the online active-learning scenario considered in this paper. Specifically, all algorithms are evaluated using the same interleaved test-then-train (prequential) evaluation protocol and operate under the same labeling budget constraint. For synthetic streams, identical generator configurations and drift settings are used for all methods, ensuring that the imbalance evolution and concept-drift patterns are consistent across algorithms. In addition, all methods are executed within the same MOA framework and follow the same preprocessing and random-seed settings whenever applicable. Because earlier studies may adopt different evaluation protocols, supervision levels, or stream configurations, the absolute numerical values reported in this paper may differ from those originally reported in the literature. Nevertheless, re-evaluating all methods under the same controlled conditions ensures that the comparisons presented here are consistent, reproducible, and fair. To reduce the influence of random fluctuations, all stochastic components are controlled by fixed random seeds, and the final reported values are averaged over repeated runs whenever the corresponding method contains random operations. This setting improves the stability of the reported comparisons and avoids drawing conclusions from a single random execution.

3.4 Comparative experiments with methods

In this subsection, we conduct a systematic comparison between DMAE and representative state of the art methods on 15 synthetic data streams and 5 real world data stream datasets. The compared datasets are reported in Tables 1 and 2, including 15 synthetic streams and 5 real streams, covering different degrees of multi class imbalance, different

Table 8 Results of parameter sensitivity experiments for θ, β, and δ

	Baseline uncertainty threshold θ				mixing weight β between BAM^2 and the baseline threshold					marginal corridor width δ in BAM^2			
	0.30	0.40	0.50	0.60	0.10	0.30	0.50	0.70	0.90	0.01	0.03	0.05	0.07
Accuracy(%)	93.98	94.28	94.46	94.32	93.98	94.22	94.46	94.14	93.95	93.86	94.09	94.46	93.97
Kappa(%)	92.03	92.24	92.50	92.26	91.92	92.18	92.50	92.25	91.97	91.56	92.12	92.50	91.67
G-Mean(%)	90.08	90.46	90.80	90.34	90.18	90.34	90.80	90.26	89.91	89.95	90.22	90.80	89.85
Recall (%)	90.02	90.25	90.59	90.32	90.05	90.39	90.59	90.20	89.98	89.78	90.25	90.59	89.63
Label(%)	19.8	19.6	19.6	19.5	19.5	19.6	19.7	19.8	19.8	19.8	19.6	19.8	19.6

Table 9 Results of parameter sensitivity experiments for ρ_{max} and λ_{long}

	upper bound of the short term memory ratio under strong drift ρ_{max}					global scaling factor of long term prototypes λ_{long}				
	0.60	0.70	0.80	0.90	0.95	0.40	0.60	0.80	1.00	1.20
Accuracy(%)	94.05	94.23	94.46	94.08	94.01	93.81	94.12	94.46	94.04	93.95
Kappa(%)	91.88	92.09	92.50	91.95	91.98	91.72	92.11	92.50	92.06	92.01
G-Mean(%)	90.28	90.32	90.80	90.29	90.34	89.96	90.22	90.80	90.36	90.03
Recall (%)	90.18	90.35	90.59	90.22	90.17	89.87	90.25	90.59	90.14	89.94
Label(%)	19.6	19.6	19.6	19.6	19.6	19.6	19.6	19.6	19.8	19.8

drift patterns, and their combinations, with the goal of comprehensively examining stability and generalization under dynamic environments. To objectively evaluate the overall performance of DMAE in scenarios where multi class imbalance and concept drift coexist, we select 10 representative baseline methods as the control group. These methods span the major technical routes for handling imbalance and drift in data stream classification, including imbalance oriented specialized ensembles, online learners centered on drift adaptation, and hybrid or active learning frameworks under labeling budget constraints, thereby ensuring broad coverage and fairness in the comparison. Below we briefly summarize the core ideas and the primary focus of each baseline. The first category consists of imbalance oriented methods, which aim to mitigate learning bias caused by the scarcity of minority class instances. The robust online self adjusting ensemble ROSE improves component diversity via random feature subspaces and dynamically maintains training information through class wise sliding windows and a self adjustment mechanism, enabling strong robustness under high imbalance ratios and gradual drifts. The multi class oversampling online bagging method MOOB introduces a time decayed oversampling strategy into the online bagging framework, increasing minority class exposure during incremental training and thus improving learnability for rare categories. CSMOTE dynamically generates synthetic minority class samples to maintain within block class balance, which can partially resist minority boundary collapse under drift, but it often incurs higher computational cost and is more sensitive to sample quality in high dimensional or complex distributions. The second category includes drift adaptive methods, whose core lies in rapid perception of distributional changes and timely model updating. The Kappa updated ensemble KUE weights ensemble members using the Kappa statistic and adjusts member contributions driven by performance feedback, which helps maintain adaptivity during transitions from stationary to drifting phases, while its feedback reliability becomes more sensitive under severe class imbalance. SRP uses a pruned reservoir to control resource usage and combines explicit drift detection to trigger updates, offering strong practicality in resource constrained streaming environments, although its mechanism is not specifically designed for multi class imbalance. The cost sensitive adaptive random forest CSARF incorporates cost sensitive learning into tree modeling and ensemble updating, leveraging class distribution information to balance overall accuracy and minority recognition ability. It is suitable for complex drift scenarios, yet it may still face adaptation pressure when abrupt drifts coincide with drastic changes in multi class structure. The third category comprises hybrid and active learning methods, which emphasize improving label utilization efficiency and overall performance under limited labeling budgets. CALMID selects high information instances via an uncertainty threshold driven querying mechanism and combines imbalance oriented strategies to enable more targeted learning, making it suitable for label scarce multi class data streams, although its effectiveness often depends on threshold settings and how class difficulty is characterized. The MicFoal framework can dynamically adjust the labeling budget according to the stream status to accommodate changes in the number of classes or fluctuations in task difficulty. AdaOB combines online bagging with dynamic resampling and strengthens minority class representation through an adaptive mechanism. OALM-MI assigns higher attention to rare events via incremental weight updating, thereby improving minority recall, but its systematic capability for drift adaptation is relatively limited. In summary, these baseline methods form a complementary comparison suite from three perspectives, imbalance correction, drift adaptation, and label efficiency, providing a solid and representative reference for validating DMAE in terms of active querying, drift responsiveness, and robustness to multi class imbalance.

To ensure fairness in the comparison, all compared ensemble methods use the Hoeffding Tree as the base classifier. The data block size is set to 500, and for active learning strategies, the number of initial instances is also set to 500. The labelling budget for all active-learning algorithms is fixed at 20%, and this budget is enforced as a global query budget throughout the online process.

3.4.1 Accuracy and kappa results statistics

Tables 3 and 4 report the Accuracy and Kappa results of all compared methods. In Tables 3 and 4, the first row lists the ten baseline algorithms and our proposed method, while

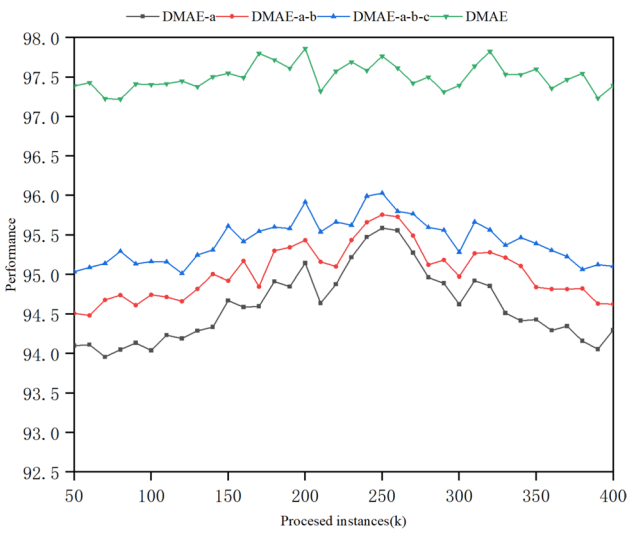


Fig. 8 Temporal ablation results of DMAE on the VarImb_SVg_2 stream

the remaining rows present the corresponding Accuracy or Kappa values of each method on the 20 data streams.

Overall, the results show that DMAE achieves competitive and generally strong performance on most datasets in handling data streams with multi class imbalance and concept drift. In terms of Accuracy, DMAE ranks first on 19 out of the 20 datasets, and is only slightly worse than SRP on the diabetes dataset. In most scenarios, DMAE also achieves a clear margin over the competing methods. For example, on the real world benchmark dataset banana, DMAE improves Accuracy by approximately 30 percent compared with CALMID, indicating that DMAE can capture minority class characteristics more effectively and reduce overall misclassification. On the synthetic dataset VarImb SIS2, DMAE exceeds MicFoal by about 41 percent in Accuracy. Moreover, DMAE attains Accuracy above 90 percent on all synthetic datasets, and exceeds 95 percent on more than half of them, indicating that DMAE maintains stable performance under noisy and imbalanced environments.

In terms of the Kappa coefficient, DMAE also performs remarkably well. Across all 20 datasets, DMAE is only slightly inferior to ROSE on the diabetes dataset, while ranking first on the remaining 19 datasets. For example, on banana, DMAE improves Kappa by approximately 43 percent compared with CALMID, suggesting that DMAE is highly suitable for multi class imbalance settings. On the VarImb SIS2 dataset, DMAE achieves a Kappa value about 16 percent higher than AdaOB, highlighting its capability to minimize false positives and enhance classification consistency under severe imbalance. In contrast, CALMID, OALM-MI, and AdaOB, as representative active learning approaches, obtain Kappa values above 85 percent on several synthetic datasets, yet their overall averages still lag behind DMAE by roughly 5 percent to even 10 percent. Moreover, on the real world dataset letter, most methods achieve Kappa values around 60 percent, whereas DMAE reaches nearly 72 percent, further emphasizing its comprehensive advantages in generalization ability and minority class detection.

3.4.2 G-mean and recall results statistics

Tables 5 and 6 report the G-Mean and Recall results of all compared methods. In Tables 5 and 6, the first row lists the ten baseline algorithms and our proposed method, while the remaining rows provide the corresponding G-Mean or Recall values of each method on the 20 data streams.

The results indicate that DMAE achieves the best G-Mean performance on 19 out of the 20 datasets, being only slightly worse than ROSE on the diabetes dataset. DMAE also surpasses other methods by a large margin in key scenarios. For instance, on the real world benchmark dataset banana, DMAE improves G-Mean by approximately 62 percent compared with CALMID, suggesting that DMAE can more effectively balance recall across classes and avoid excessive bias toward the majority class in imbalanced environments. On the synthetic dataset VarImb SIS2, DMAE exceeds MicFoal by about 34 percent in G-Mean. Across all synthetic datasets, DMAE outperforms representative methods for imbalanced drifting streams such as CALMID, ROSE, and OALM-MI by more than 5 percent on average, and compared with weaker baselines such as MicFoal, CSMOTE, and CSARF, the improvement can even be more than doubled. These advantages further demonstrate DMAE's enhanced balanced classification capability under noisy variants and its increased sensitivity to minority classes.

For the Recall metric, DMAE also delivers consistently strong performance, achieving the best rank across all datasets. For example, on letter, most methods obtain Recall values around 70 percent or even lower, whereas DMAE reaches approximately 82 percent, indicating more accurate minority class detection and making it particularly suitable for dynamically drifting stream scenarios. On the VarImb SIS2 dataset, DMAE improves Recall by about 14 percent compared with AdaOB. On the VarImb ISSI dataset, DMAE attains a Recall of 98.17 percent, which is roughly 17 percent higher than the representative method ROSE. This highlights DMAE's ability to minimize missed detections

Table 11 Results of the iman and davenport tests

Indicators	Iman–Davenport values	Critical value ($\alpha = 0.05$)	p-values	Significant differences?
Accuracy	70.22	1.881	=2.26e-58	Yes
Kappa	71.89	1.881	=3.94e-59	Yes
G-Mean	73.68	1.881	=6.29e-60	Yes
Recall	41.83	1.881	=8.44e-43	Yes

Table 10 Average algorithm ranks used in the iman and davenport tests

	DMAE	OALM-MI	CALMID	ROSE	AdaOB	SRP	MOOB	KUE	MicFoal	CSARF	CSMOTE
Accuracy	1.05	3.80	3.48	4.78	4.40	5.05	7.55	7.00	8.40	9.70	10.80
Kappa	1.10	4.05	3.65	4.35	4.20	5.12	7.42	6.98	8.40	10.10	10.62
G-Mean	1.10	3.80	4.25	4.35	3.40	5.90	6.70	7.45	8.40	9.95	10.70
Recall	1.00	4.85	3.45	4.20	4.50	5.90	7.45	6.50	8.75	9.80	9.60

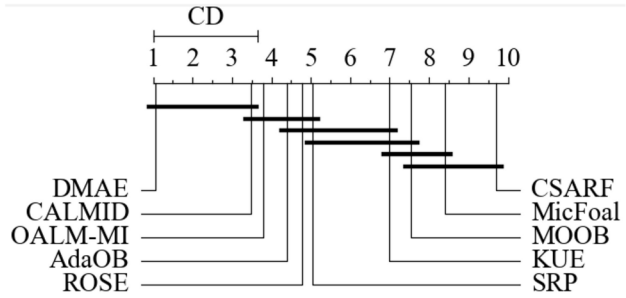


Fig. 9 Nemenyi test with 95% confidence level on accuracy

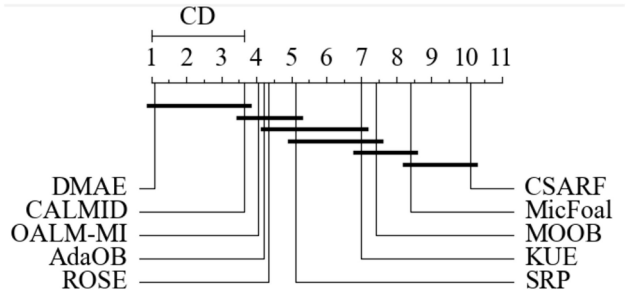


Fig. 10 Nemenyi test with 95% confidence level on kappa

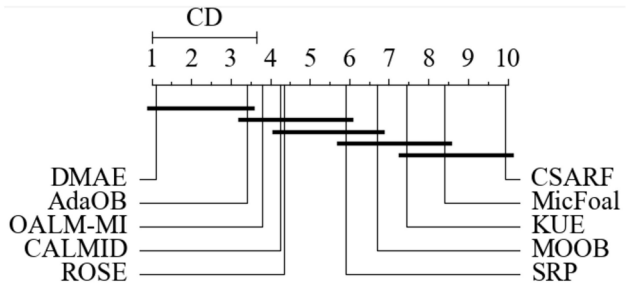


Fig. 11 Nemenyi test with 95% confidence level on G-mean

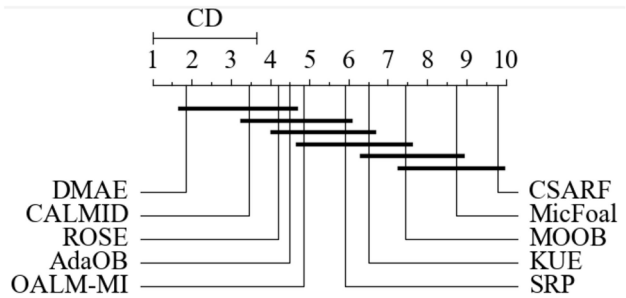


Fig. 12 Nemenyi test with 95% confidence level on recall

and optimize minority class recognition under severely imbalanced synthetic streams, while reducing the potential risk of false negatives. From the overall Recall results, most methods achieve Recall values between 80 and 90 percent on the synthetic datasets, whereas DMAE remains above 90 percent throughout, and exceeds 95 percent on half of them. These results not only reflect the strong generalization, multi class balance, and robustness of DMAE, but also verify its high effectiveness in both synthetic and real world settings.

3.5 Comprehensive metrics performance analysis

To evaluate the performance of the algorithms throughout the learning process, we use the average of four evaluation metrics, namely Accuracy, Kappa, G-Mean, and Recall, to present the final real time performance. The aggregated metric is computed once every 1000 instances. In this study, we compare methods on 20 datasets, and we only plot the aggregated metric curves on representative and relatively complex datasets to highlight the overall advantages of our method. Figure 2 shows the aggregated metric curves on eight synthetic data streams.

Across the streams shown in Fig. 2, the aggregated metric curves suggest that DMAE obtains generally strong overall performance compared with most competing methods. When concept drift occurs, the aggregated performance curves of many methods first exhibit a decline and then gradually recover. As shown in Figs. 2c, 2f, and 2 g, when an abrupt drift occurs around 100 k instances, the performance of most methods decreases noticeably. DMAE shows a relatively faster recovery trend and maintains a higher aggregated metric after recovery in these representative cases. In addition, as shown in Figs. 2a, 2b, and 2 h, under gradual drift, the performance degradation of DMAE is generally smaller than that of most compared algorithms. For incremental drift, as illustrated in Figs. 2a, 2 d, 2e, and 2f, DMAE also shows stable recovery behavior after performance degradation. Overall, the results in Fig. 2 indicate that DMAE can adapt effectively to different types of concept drift and changes in multiclass imbalance ratios under a limited labelling budget.

In addition to the overall performance comparison, Fig. 2 also illustrates how the predictive performance of different algorithms evolves during the online learning process. The horizontal axis represents the number of processed instances, while the vertical axis shows the corresponding aggregated metric. The locations of concept drift events are explicitly marked in the figure, including sudden drift, gradual drift, virtual drift, and recurrent drift.

From the temporal curves in Fig. 2, it can be observed that when concept drift occurs, the predictive performance of most algorithms temporarily decreases due to the mismatch between the previously learned model and the newly emerging data distribution. Compared with several competing methods, DMAE shows better recovery behavior and tends to stabilize at a relatively high level after drift. This behavior suggests that the dual-memory mechanism and adaptive ensemble updating strategy help the model absorb new information while maintaining stable performance throughout the streaming process. Overall, the temporal behavior shown in Fig. 2 provides evidence that DMAE can maintain competitive average performance and relatively stable adaptation when facing different types of concept drift in evolving data streams.

3.6 Overall multi metric comparison and average rank analysis

This subsection presents a comprehensive comparison of the competing algorithms from two perspectives: multiple evaluation metrics and different types of data streams. The comparison is conducted using Accuracy, Kappa, G-Mean, and Recall on both synthetic and real data streams. To provide a clear and interpretable analysis, the results are summarized using average-rank statistics, wind-rose visualizations of multi-metric rankings, and ridge plots of performance distributions across datasets.

Table 7 reports the average ranks of all algorithms on the synthetic streams, the real-world streams, and the combined set of streams for each metric. The table further summarizes the overall relative performance of each method using a meta-ranking across metrics. As shown in Table 7, DMAE obtains the best or near-best rank across the four evaluation metrics on both synthetic and real data streams. This result suggests that the proposed framework maintains strong predictive performance while preserving balanced recognition capability under multiclass imbalance and concept drift.

Figure 3 provides a visual summary of the ranking patterns reported in Table 7. In this figure, each sector corresponds to one algorithm, and the sectors are arranged clockwise. The four colored radial bands within each sector represent the four evaluation metrics, namely Accuracy, Kappa, G-Mean, and Recall. The radial extent encodes the relative ranking performance of the algorithm on the corresponding metric for the given stream type, where a larger outward extent indicates a better ranking position after consistent normalization for visualization. Figure 3a summarizes the results on the synthetic streams, and Fig. 3b summarizes the results on the real-world streams. This wind-rose representation provides a compact visualization of the multi-metric ranking profile of each algorithm and highlights whether a method maintains balanced performance across different metrics. From both subfigures, DMAE shows a favorable ranking profile across most metrics, indicating that it performs competitively compared with the baseline approaches.

While the ranking analysis summarizes the relative performance of the algorithms, it does not reveal the distributional behavior of the performance scores across datasets. To further examine the stability and variability of the compared methods, we analyze the distribution of the metric values using ridge plots.

Figures 4, 5, 6 and 7 present ridge plots for Accuracy, Kappa, G-Mean, and Recall on the 15 synthetic datasets. In these plots, the horizontal axis represents the value of the corresponding evaluation metric, the vertical axis lists the compared algorithms, and the height of each ridge curve represents the kernel density of the metric values obtained across the datasets. A ridge curve shifted further to the right indicates that the corresponding algorithm tends to achieve higher metric values overall, whereas a narrower and more concentrated ridge indicates smaller cross-dataset variation. From Figures 4, 5, 6 and 7, visible differences can be observed in the performance distributions of the compared methods. The ridge curves of DMAE are generally located toward the high-value region across the four evaluation metrics, suggesting that DMAE achieves favorable performance on most synthetic datasets. In terms of distribution shape, DMAE also shows a relatively concentrated pattern compared with several baseline methods, indicating that its cross-dataset performance fluctuation is comparatively controlled. By contrast, some baseline methods exhibit wider or more dispersed ridge shapes, suggesting that their performance may be more sensitive to specific drift types or imbalance patterns. More importantly, the ridge curves corresponding to Accuracy, Kappa, G-Mean, and Recall show similar right-shifted patterns for DMAE. This consistency suggests that the performance gain of DMAE is not limited to a single metric, but is reflected across predictive accuracy, agreement-based evaluation, balanced recognition, and recall-oriented assessment.

Overall, the ridge-plot analysis suggests that DMAE achieves a relatively high central performance level and controlled cross-stream variation compared with most competing algorithms. Taken together, Table 7, Fig. 3, and Figures 4, 5, 6 and 7 provide complementary evidence regarding the performance behavior of DMAE. Table 7 quantifies the relative ranking of the algorithms, Fig. 3 summarizes the multi-metric ranking profile, and the ridge plots illustrate the distributional behavior and stability of the algorithms across datasets. To further examine whether the observed performance differences are statistically reliable, we next perform non-parametric statistical tests across all algorithms.

3.7 Parameter sensitivity analysis

To examine the parameter sensitivity of the proposed DMAE, this subsection reports the results of a set of parameter sensitivity experiments. The basic idea is to vary one target parameter while keeping all other parameters unchanged. For a fair comparison and to keep the computational cost controllable, the ensemble size of all ensemble methods is fixed to 10. The investigated parameters include the baseline uncertainty threshold θ, the mixing weight β between BAM^2 and the baseline threshold, the marginal corridor width δ in BAM^2, the upper bound of the short term memory ratio under strong drift ρ_{max}, and the global scaling factor of long term prototypes λ_{long}. The parameter sensitivity experiments are conducted on the synthetic data stream VarImb GIS-GrN10 5. The default parameter setting is θ equals 0.50, β equals 0.50, δ equals 0.05, ρ_{max} equals 0.80, and λ_{long} equals 0.80. Tables 8 and 9 present the results of the parameter sensitivity experiments.

As shown in the tables, in data stream environments with multi class imbalance accompanied by concept drift, DMAE exhibits strong overall robustness with respect to the five key hyperparameters. By considering Tables 8 and 9 together, it can be seen that when adjusting the baseline uncertainty threshold, the BAM^2 mixing weight and corridor width, as well as the upper bound of the short term memory ratio and the long term prototype scaling factor in SDMI within the preset ranges, Accuracy, Kappa, G-Mean, and Recall all remain at a high level with only limited fluctuations, indicating that the performance is mainly determined by the proposed structural design rather than depending on delicate hyperparameter tuning. For the baseline uncertainty threshold theta, we observe that when theta increases gradually from a smaller value to a moderate level, the four metrics generally show a trend of improving first and then stabilizing, reaching a relatively high level around the moderate threshold; when theta continues to increase, the metrics only weaken slightly, suggesting that neither overly aggressive nor overly conservative querying strategies are advantageous, while a moderate threshold achieves a more reasonable balance between the number of labeled samples and their difficulty, so that the selected instances provide sufficient information gain without substantially increasing labeling cost, which is consistent with the proposed margin based active learning rationale. For the BAM^2 mixing weight beta, as beta increases from weak intervention to a moderate strength, G-Mean and Recall, especially the minority class related metrics, improve more noticeably, indicating that within this range, threshold correction guided by class pair statistics and drift intensity can effectively enhance discrimination near the decision boundary; when beta further increases, the four metrics decrease slightly but the overall level remains high, implying that overly strong correction may weaken the stabilizing role of the original threshold, yet it does not disrupt the overall performance profile. The variation trend of the marginal corridor width delta reflects the necessity of activating BAM^2 only near the decision boundary: when the corridor expands from extremely narrow to a moderate width, the four metrics rise simultaneously, suggesting that more truly ambiguous samples receive class pair calibration; when delta continues to widen, performance drops slightly, indicating that frequent correction on high confidence samples provides no additional benefit and may instead introduce mild fluctuations. The results for the upper bound of the short term memory ratio under strong drift, rho max, show that increasing rho max from a small value to a moderate level leads to faster responses to abrupt concept changes, with clear improvements in G-Mean and Recall; when rho max becomes even larger, the metrics tend to level off or decline slightly, indicating that excessive reliance on the most recent window weakens the utilization of periodic and stable patterns. The long term prototype scaling factor lambda long reflects the regularization strength of long term memory: when lambda long increases from a small value to a moderate level, Accuracy and Kappa improve continuously and the decision boundary becomes smoother and more stable; when lambda long is too large, the improvement in classification performance becomes limited while the querying ratio rises markedly, suggesting that overly strong long term constraints reduce the efficiency of absorbing new concepts. Overall, these five parameters follow a consistent pattern in which moderate values are generally better, while the two extremes are slightly weaker but the overall performance remains stable, and the observed trends match the expected behavior of the localized threshold correction mechanism in BAM^2 Lite and the dual memory design in SDMI. In general, the performance changes only marginally when tuning these parameters, therefore DMAE is not particularly sensitive to their settings.

3.8 Ablation study on the components of DMAE

To examine the contribution of the major components of DMAE, we conduct a progressive component-level ablation study on the core modules of the framework. Instead of changing the base classifier or the evaluation protocol, all variants are evaluated under the same online active-learning setting, the same labelling budget, and the same data stream. Starting from a simplified active-learning-based online ensemble, we gradually add the main components of DMAE and observe how the online performance changes over time. This design provides a direct way to assess whether each major module brings additional benefits when it is incorporated into the framework. The compared variants are

defined as follows. DMAE-a denotes the base active learner, which removes BAM^2-Lite, SDMI-PDSR, and PCN-Weight, and retains only the basic active online ensemble update. DMAE-a-b adds BAM^2-Lite to DMAE-a, so as to examine the effect of the hybrid uncertainty estimation and querying mechanism. DMAE-a-b-c further adds SDMI-PDSR, which introduces drift-strength-aware dual-memory initialization and trial-based soft replacement. DMAE denotes the full model, which additionally includes PCN-Weight for comprehensive sample weighting. In this way, the ablation study focuses on the incremental effect of three major components: BAM^2-Lite -Lite, SDMI-PDSR, and PCN-Weight.

All variants are evaluated on the synthetic data stream VarImb_SVg_2, which contains both evolving class imbalance and concept drift. This stream is suitable for observing whether each component contributes to online adaptation under dynamic imbalance. Figure 8 reports the temporal performance curves of the four variants as the number of processed instances increases. As shown in Fig. 8, DMAE-a obtains the lowest performance among the four variants in most stages of the stream. This result indicates that a basic active online ensemble is not sufficient to handle multiclass imbalance and concept drift under a limited labelling budget. After incorporating BAM^2-Lite, DMAE-a-b shows a visible improvement over DMAE-a. This suggests that the hybrid uncertainty and querying mechanism helps allocate limited labels to more informative boundary regions, thereby improving the quality of labelled samples used for online updating. When SDMI-PDSR is further introduced, DMAE-a-b-c achieves higher and relatively more stable performance than DMAE-a-b, especially around the middle part of the stream where the performance curves show more fluctuation. This observation suggests that drift-strength-aware dual-memory initialization and trial-based soft replacement can help the ensemble adapt after distributional changes and reduce the negative effect of abrupt model replacement. Finally, the full DMAE model achieves the highest curve among the four variants in most stages. This additional improvement indicates that PCN-Weight further enhances the use of queried samples by assigning stronger training emphasis to difficult and minority-relevant instances.

Overall, the progressive ablation results in Fig. 8 show that the three major components of DMAE are complementary rather than redundant. BAM^2-Lite mainly improves annotation efficiency and boundary-focused querying; SDMI-PDSR contributes to drift response and ensemble evolution; and PCN-Weight further strengthens weighted adaptation to hard and minority-relevant samples. Although this experiment focuses on component-level ablation rather than isolating every internal operation of each module, the consistent performance gains across the progressive variants provide evidence that the main modules jointly contribute to the final performance of DMAE under drifting multiclass imbalanced data streams.

3.8.1 Statistical significance analysis

This subsection reports the statistical significance analysis based on the Accuracy, Kappa, G-Mean, and Recall results of all methods on the 20 experimental data streams, involving a total of 11 algorithms. For each dataset, the performance values of the compared algorithms are first obtained from the online evaluation process and then converted into ranks. The statistical analysis is therefore conducted on the original per-dataset performance values rather than on aggregated averages, ensuring that the variability across datasets is properly considered in the statistical inference. The average ranks of all algorithms across the datasets with respect to the four evaluation metrics are summarized in Table 10, where a lower average rank indicates better overall performance. As shown in the table, DMAE consistently achieves the best average rank across all metrics, whereas CSMOTE ranks last. To determine whether the observed differences are statistically significant rather than caused by random variations across datasets, we adopt a non-parametric statistical testing framework commonly used for algorithm comparison across multiple datasets (Demšar 2006). Compared with parametric tests, non-parametric tests do not assume a specific distribution of the performance values and are therefore more suitable for evaluating multiple algorithms under heterogeneous data-stream environments.

Following this framework, we first apply an omnibus significance test to examine whether there exist overall performance differences among the compared algorithms. Although the Friedman test is the classical non-parametric approach for multi-algorithm comparison, it is often considered relatively conservative when the number of algorithms and datasets is limited. Therefore, we employ the Iman–Davenport test, which can be regarded as an improved version of the Friedman test and provides a more accurate F-distribution approximation while preserving the same rank-based comparison principle. The null hypothesis assumes that there is no significant performance difference among the compared algorithms. If the null hypothesis is rejected, we further conduct the Nemenyi post-hoc test to determine whether the performance of DMAE differs significantly from that of the other competing methods.

Table 11 reports the results of the Iman Davenport test. At the 0.05 significance level, the critical value is obtained from the F distribution with 10 and 190 degrees of freedom. For Accuracy, Kappa, G-Mean, and Recall, the Iman Davenport statistics are all larger than their corresponding critical values, indicating that the differences in algorithm ranks are statistically significant. We then

perform the Nemenyi post hoc test, where the critical difference is 2.655. To keep the critical difference diagrams readable and to focus on the most competitive methods, we present only the top ten ranked algorithms, since these methods show stronger adaptability, faster recovery, and more stable performance than CSMOTE across a variety of concept drift settings, and including CSMOTE would provide limited additional insight while reducing figure clarity. Figures 9, 10, 11, and 12 show the Nemenyi test results for Accuracy, Kappa, G-Mean, and Recall, respectively. The diagrams indicate that DMAE is not significantly different from OALM-MI, whereas it significantly outperforms the other nine algorithms.

4 Conclusions

This study addresses the coupled challenge of multiclass imbalance and concept drift in online active learning. DMAE is designed to coordinate budgeted uncertainty querying, drift-aware memory reconstruction, soft ensemble replacement, and weighted online updating within a unified framework. From the perspective of uncertainty modeling, DMAE decomposes the conventional single margin-based querying rule into a global baseline threshold and a class-pair-level adaptive correction provided by BAM^2-Lite. The baseline threshold preserves the general structure of the original decision rule, while the class-pair correction is activated within a predefined margin corridor and adjusts the effective threshold according to class-pair information, imbalance structure, and drift intensity. This design encourages the model to query informative boundary samples, especially those related to minority classes, while avoiding excessive label consumption on high-confidence instances. For drift adaptation, the SDMI module balances short-term and long-term memory according to the estimated drift strength. When the drift signal is weak, the candidate classifier inherits more information from long-term prototypes so that relatively stable historical patterns can be preserved. When the drift signal becomes stronger, the short-term window receives more emphasis, allowing the candidate classifier to adapt more quickly to the newly emerging distribution. Combined with the PDSR-based trial replacement strategy, DMAE avoids directly switching between full model reset and completely fixed ensemble structures. Instead, it evaluates candidate classifiers before replacement, thereby improving the stability of ensemble evolution under changing data distributions. For online updating, PCN-Weight connects the queried samples with the incremental training process by assigning larger weights to hard, minority-relevant, and structurally informative instances. In this way, samples emphasized during the querying stage can also receive sufficient attention during model updating. This mechanism helps reduce the mismatch between active query selection and imbalance-aware learning, which is a common issue in multiclass imbalanced data streams under limited labels. Experimental results on synthetic and real-world data streams show that DMAE achieves competitive and generally stable performance compared with representative sampling methods, cost-sensitive ensemble methods, drift-adaptive methods, and active-learning baselines. The results on multiple metrics, average ranks, ablation analysis, parameter sensitivity analysis, and statistical tests indicate that the proposed components contribute to improving label utilization, minority-class recognition, and adaptation to evolving distributions. These findings suggest that jointly modeling uncertainty, memory, and sample weighting is beneficial for online active learning in multiclass imbalanced concept-drifting data streams. This study also has several limitations. First, DMAE relies on random-query labels in the label sliding window to estimate the current class distribution. When the labelling budget is extremely small, this estimate may become less stable, especially for rare classes. Second, although the memory sizes and PDSR trial length are bounded, frequent drift events may still introduce additional computational overhead due to repeated candidate initialization and trial evaluation. Third, the current framework is mainly evaluated with traditional online ensemble learners, and its behavior with deep streaming models or more complex feature representations requires further investigation. Finally, the present experiments focus on component-level ablation, while more fine-grained analysis of the internal operations of each module could provide deeper insight into the contribution of each design choice.

Future work will focus on improving the efficiency and adaptability of DMAE in more demanding online environments. One direction is to further reduce the computational overhead of BAM^2-Lite and SDMI-PDSR, so that the method can better fit high-frequency and resource-constrained streaming scenarios. Another direction is to develop more adaptive budget allocation strategies, allowing the labelling budget to be adjusted according to drift severity, class imbalance, and model uncertainty. It is also valuable to extend the decomposable asymmetric margin and dual-memory principles to more complex tasks, such as multilabel learning, graph-structured data streams, and deep streaming models. These extensions may further improve the applicability of uncertainty-aware active ensemble learning in real-world dynamic environments.

Acknowledgements The authors would like to thank the anonymous reviewers and the editors for their valuable comments and suggestions. Ethics approval: Not applicable. This study did not involve human participants or animals. Informed consent:Not applicable. Data availability:The datasets used in this study are publicly available. The

code and materials are available from the corresponding author upon reasonable request. Author contributions:Yajie Xue: Conceptualization, Methodology, Software, Investigation, Writing—original draft. Meng Han: Supervision, Funding acquisition, Writing—review & editing. Yikai Li: Validation, Experiments, Data curation. Cuicui Ma: Visualization, Formal analysis. Jian Ding: Resources, Validation. Juan Li: Writing—review & editing.

Author contribution Yajie Xue and Meng Han conceived the study and designed the method. Yajie Xue implemented the model, conducted the experiments, analyzed the results, and wrote the main manuscript text. Meng Han supervised the research and revised the manuscript. Yikai Li and Cuicui Ma assisted with data processing, experimental evaluation, and result analysis. Jian Ding and Juan Li contributed to methodology discussion, manuscript revision, and language polishing. All authors reviewed the manuscript and approved the final version.

Funding This research was supported by the National Natural Science Foundation of China (62566001), the Natural Science Foundation of Ningxia (2025AAC030054), the Natural Science Foundation of Ningxia (2026AAC030395), the Central Universities Foundation of North Minzu University (2021KJCX10).

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare that they have no know competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Brzezinski D, Stefanowski J, Susmaga R et al (2020) On the dynamics of classification measures for imbalanced and streaming data. IEEE Trans Neural Netw Learn Syst 31(8):2868–2878. https://doi.org/10.1109/TNNLS.2019.2939695

Cano A, Krawczyk B (2022) ROSE: Robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams. Mach Learn 111(7):2561–2599. https://doi.org/10.1007/s10994-021-06034-2

Chen S, He H (2011) Towards incremental learning of nonstationary imbalanced data stream: A multiple selectively recursive approach. Evol Syst 2(1):35–50. https://doi.org/10.1007/s12530-011-9038-y

Chen H, Liu H, Wu Z, Fan X, Cao L (2025) FedSI: federated subnetwork inference for efficient uncertainty quantification. IEEE Trans Neural Netw Learn Syst 36(9):16975–16989

Chen H, Fan X, Wu Z, Cao L (2026) FigBO: a generalized acquisition function framework with look-ahead capability for Bayesian optimization. Mach Learn 115:110

Chen H, Fan X, Liu H, Li Y, Zhao Z, Zhou F, Quinn CJ, Cao L (2026) Federated neural nonparametric point processes. Artif Intell. https://doi.org/10.1016/j.artint.2025.104454

Czarnowski I (2022) Weighted ensemble with one-class classification and over-sampling and instance selection (WECOI): an approach for learning from imbalanced data streams. J Comput Sci 61. https://doi.org/10.1016/j.jocs.2022.101614

Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. J Mach Learn Res 7:1–30. https://doi.org/10.5555/1248547.1248548

Du H, Zhang Y, Gang K et al (2021) Online ensemble learning algorithm for imbalanced data stream. Appl Soft Comput 107. https://doi.org/10.1016/j.asoc.2021.107378

García S, Zhang ZL, Altalhi A et al (2018) Dynamic ensemble selection for multi-class imbalanced datasets. Inf Sci 445:22–37. https://doi.org/10.1016/j.ins.2018.02.053

Gong JM, Dong MG (2024) Entropy-based dynamic ensemble classification algorithm for imbalanced data stream with concept drift. PLoS ONE 19(12):e0311133. https://doi.org/10.1371/journal.pone.0311133

Guo Y, Pu J, Jiao B et al (2024) Online semi-supervised active learning ensemble classification for evolving imbalanced data streams. Appl Soft Comput 155. https://doi.org/10.1016/j.asoc.2024.111452

Halder B, Hasan KMA, Amagasa T et al (2023) Autonomic active learning strategy using cluster-based ensemble classifier for concept drifts in imbalanced data stream. Expert Syst Appl 231. https://doi.org/10.1016/j.eswa.2023.120578

Han M, Li C, Meng F et al (2024) An adaptive active learning method for multiclass imbalanced data streams with concept drift. Appl Sci (Basel) 14(16). https://doi.org/10.3390/app14167176

Hidalgo JIG, Santos SGTC, Barros RSM (2021) Dynamically adjusting diversity in ensembles for the classification of data streams with concept drift. ACM Trans Knowl Discov Data 16(2):1–20. https://doi.org/10.1145/3468913

Jian L, Shao K, Liu Y et al (2024) OEC: an online ensemble classifier for mining data streams with noisy labels. Data Min Knowl Discov 38(3):1101–1124. https://doi.org/10.1007/s10618-023-00955-2

Jiao B, Guo Y, Gong D et al (2024) Dynamic ensemble selection for imbalanced data streams with concept drift. IEEE Trans Neural Netw Learn Syst 35(1):1278–1291. https://doi.org/10.1109/TNNLS.2022.3183120

Korycki Ł, Krawczyk B (2021) Concept drift detection from multiclass imbalanced data streams. In: 2021 IEEE 37th International Conference on Data Engineering (ICDE), pp. 1068–1079. https://doi.org/10.1109/ICDE51330.2021.00098.

Krawczyk B, & Cano A (2019). Adaptive ensemble active learning for drifting data stream mining. In Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI) (pp. 2763–2771). https://doi.org/10.24963/ijcai.2019/383.

Krawczyk B, Cano A (2018) Online ensemble learning with abstaining classifiers for drifting and noisy data streams. Appl Soft Comput 68:677–692. https://doi.org/10.1016/j.asoc.2017.12.008

Liu W, Zhang H, Ding Z et al (2021) A comprehensive active learning method for multiclass imbalanced data streams with concept drift. Knowl Based Syst 215. https://doi.org/10.1016/j.knosys.2021.106778

Lu Y, Cheung YM, Tang YY (2020) Adaptive chunk-based dynamic weighted majority for imbalanced data streams with concept drift.

IEEE Transact Neur Net Learn Syst 31(8):2764–2778. https://doi.org/10.1109/TNNLS.2019.2920609

Lu Y, Cheung YM, & Tang YY (2017). Dynamic weighted majority for incremental learning of imbalanced data streams with concept drift. In Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI) (pp. 2393–2399). https://doi.org/10.24963/ijcai.2017/333.

Mahdi OA, Pardede E, Ali N (2021) A hybrid block-based ensemble framework for the multi-class problem to react to different types of drifts. Clust Comput 24(3):2327–2340. https://doi.org/10.1007/s10586-020-03157-y

Mirza B, Lin Z (2016) Meta-cognitive online sequential extreme learning machine for imbalanced and concept-drifting data classification. Neural Netw 80:79–94. https://doi.org/10.1016/j.neunet.2016.04.008

Mirza B, Lin Z, Liu N (2015) Ensemble of subset online sequential extreme learning machine for class imbalance and concept drift. Neurocomputing 149:316–329. https://doi.org/10.1016/j.neucom.2014.03.075

Nguyen VL, Shaker MH, Hüllermeier E (2022) How to measure uncertainty in uncertainty sampling for active learning. Mach Learn 111(1):89–122. https://doi.org/10.1007/s10994-021-06003-9

Niu Z, Guo W, Xue J et al (2023) 2023) A novel anomaly detection approach based on ensemble semi-supervised active learning (ADESSA. Comput Secur 129:103190. https://doi.org/10.1016/j.cose.2023.103190

Park CH, & Kang Y (2016). An active learning method for data streams with concept drift. In 2016 IEEE International Conference on Big Data (Big Data) (pp. 746–752). IEEE. https://doi.org/10.1109/BigData.2016.7840666.

Polyzos KD, Lu Q, Giannakis GB (2024) Weighted ensembles for adaptive active learning. IEEE Trans Signal Process. https://doi.org/10.1109/TSP.2024.3372401

Salama M, Abdelkader H (2022) Abdelwahab A (2022) A novel ensemble approach for heterogeneous data with active learning. Int J Eng Busi Manag 14:18479790221082604. https://doi.org/10.1177/18479790221082605

Salehi A, Khedmati M (2025) Hybrid clustering strategies for effective oversampling and undersampling in multiclass classification. Sci Rep 15(1):3460. https://doi.org/10.1038/s41598-025-86134-2

Tarazodar H, Bagherifard K, Nejatian S, & Parvin H (2024). Mitigating concept drift in data streams: an incremental decision tree approach. Soft Computing, 1–30. https://doi.org/10.1007/s00500-024-09684-2.

Wang Z, & Wang S (2023). Online automated machine learning for class imbalanced data streams. In 2023 International Joint Conference on Neural Networks (IJCNN) (pp. 1–8). IEEE. https://doi.org/10.1109/IJCNN54540.2023.10191834.

Wang S, Minku LL, & Yao X (2016). Dealing with multiple classes in online class imbalance learning. In Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI) (pp. 2118–2124). https://doi.org/10.5555/3060832.3060902.

Wu H, Han M, Chen Z et al (2023) A weighted ensemble classification algorithm based on nearest neighbors for multi-label data stream. ACM Trans Knowl Discov Data 17(5):1–21. https://doi.org/10.1145/3580483

You D, Xiao J, Wang Y et al (2023) Online learning from incomplete and imbalanced data streams. IEEE Trans Knowl Data Eng 35(10):10650–10665. https://doi.org/10.1109/TKDE.2023.3250472

Zhai J, Qi J, Shen C (2022) Binary imbalanced data classification based on diversity oversampling by generative models. Inf Sci 585:313–343. https://doi.org/10.1016/j.ins.2021.11.058

Zhang H, Liu W, Liu Q (2022) Reinforcement online active learning ensemble for drifting imbalanced data streams. IEEE Trans Knowl Data Eng 34(8):3971–3983. https://doi.org/10.1109/TKDE.2020.3026196

Zhang S, Liu K, Xu T et al (2023) A meta-heuristic feature selection algorithm combining random sampling accelerator and ensemble using data perturbation. Appl Intell 53(24):29781–29798. https://doi.org/10.1007/s10489-023-05123-0

Zhao H, Zhao C, Zhang X et al (2023) An ensemble learning approach with gradient resampling for class-imbalance problems. INFORMS J Comput 35(4):747–763. https://doi.org/10.1287/ijoc.2023.1274

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.